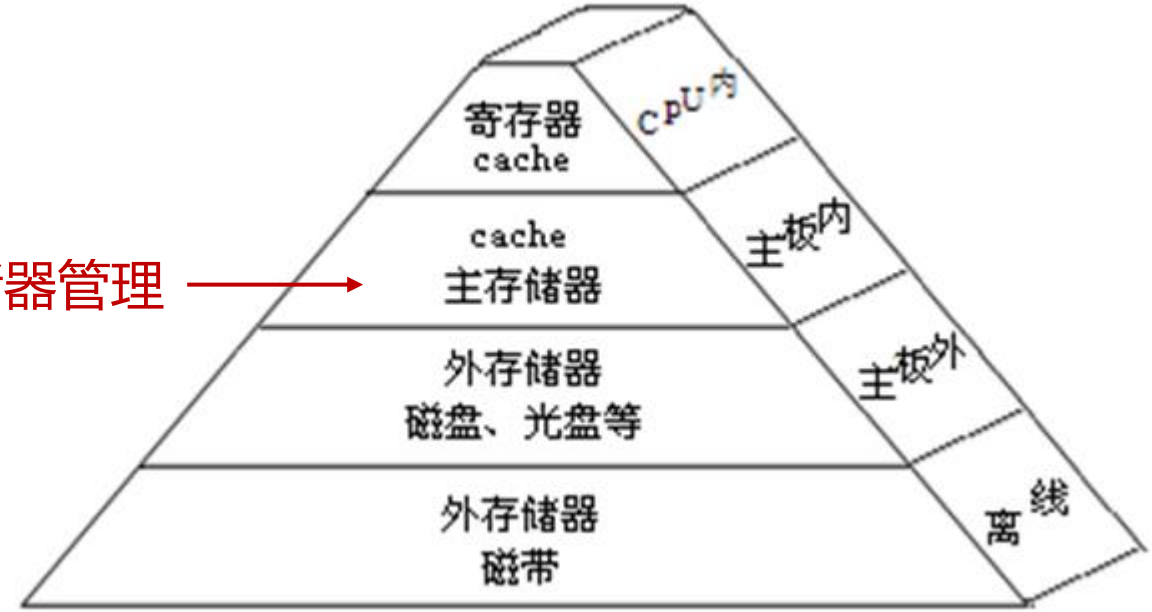
计算机操作系统

第5章 存储器管理(1)

存储器的类型

容量小 速度快



存储器管理 →

容量大 速度慢

虚拟地址和物理地址

- 程序在运行时被装入到内存中，其中的变量或指令的位置用地址值表示，地址值的表现形式可分为：
  - 虚拟（逻辑）地址：变量或指令在程序中的编号（位置）
  - 物理地址：变量或指令在内存中的位置

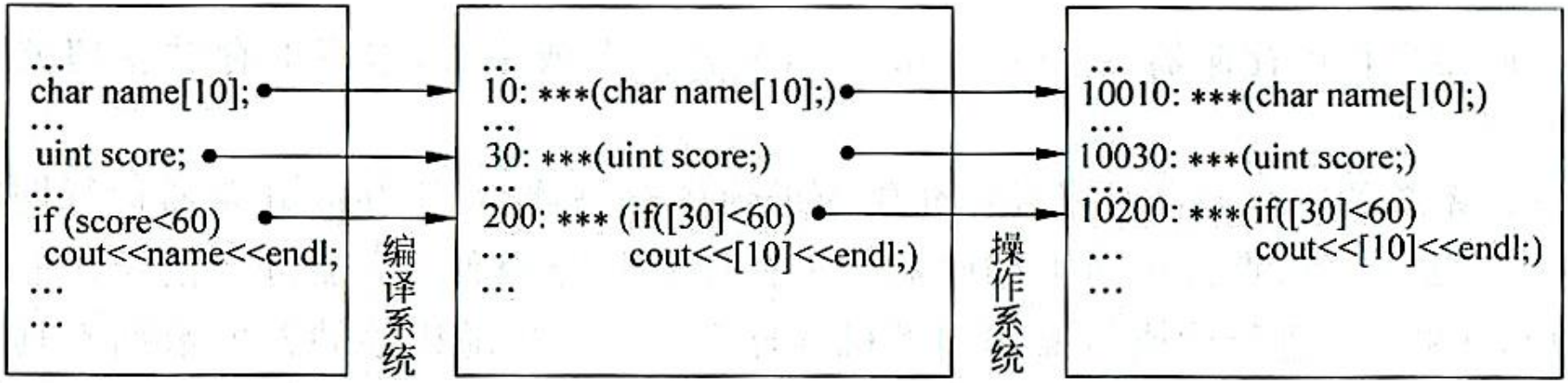


图 5-2　虚拟地址、物理地址的例子

重定位(地址转换)

- 若程序中的变量或指令用逻辑地址加以引用，则为了在运行时正确访问这些变量或指令，需要进行重定位（或称地址转换），以确定实际的物理地址。
- 根据地址转换操作的时机，重定位分为：
  - 静态重定位：装入程序时修改其中所引用的逻辑地址值为物理地址值——由操作系统实现
  - 动态重定位：在运行过程中需要做访问操作时才进行地址转换——由CPU的存储管理单元（MMU）自动实现，操作系统只登记MMU进行重定位时需要的相关信息。

重定位(地址转换)

- 静态重定位与动态重定位的特点对比

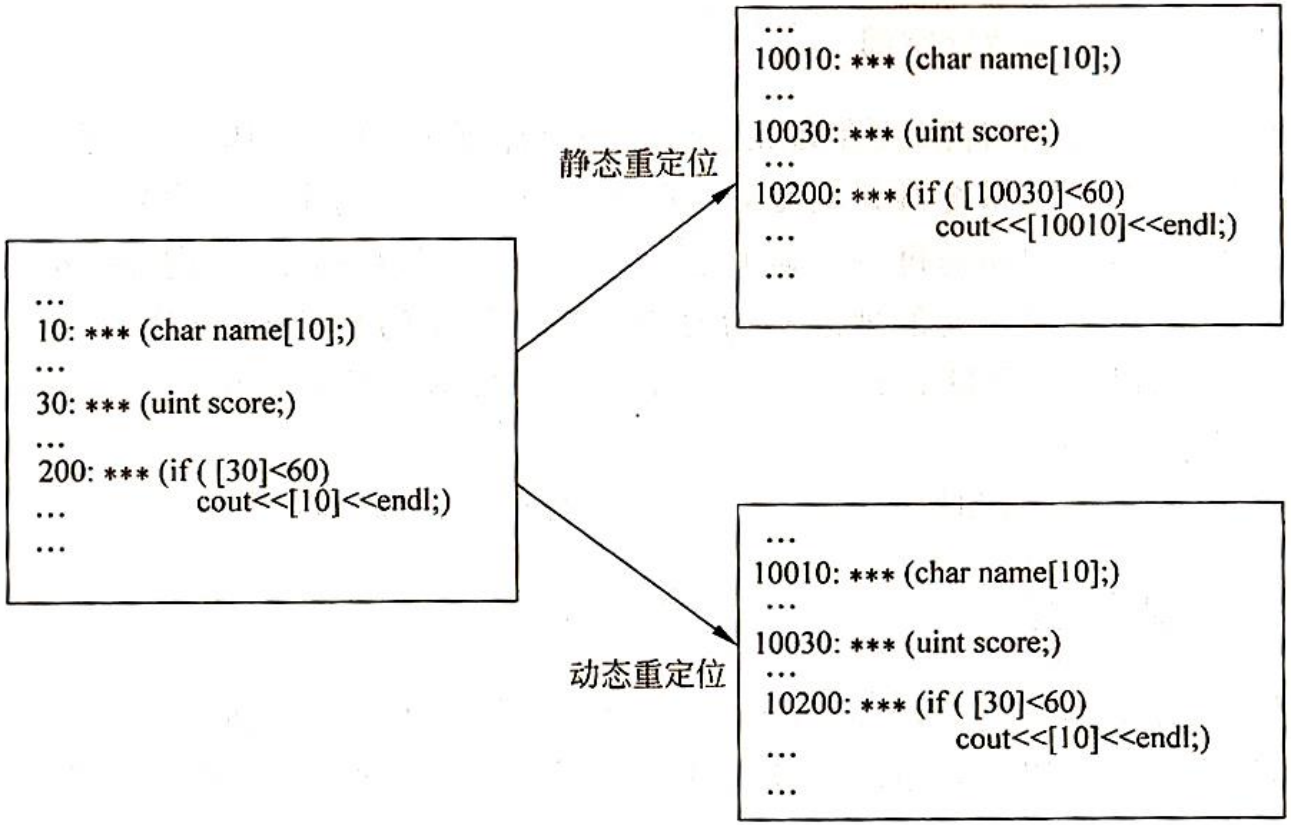


图 5-3　两种重定位方式的例子

重定位(地址转换)

❖ 静态重定位与动态重定位的特点对比

	静态重定位	动态重定位
硬件支持	不需要	需要
操作系统的工作	装入时修改程序	登记MMU所需信息
程序装入	被修改	直接装入
程序运行时移动	困难	容易
程序的不连续装入	困难	容易
其他技术支持 (动态链接、虚拟存储 等)	难以实现	容易实现

存储器管理的目的和功能

- 存储器管理主要是对用户区进行管理，其目的是提高主存储器的利用率，方便用户对主存储器的使用。
- 存储器管理中的主要功能包括：
  - 1. 存储空间的分配与回收
    - 设计合理的数据结构以记录存储空间的分配状况
    - 设计合理的算法以提高存储器利用率（装入尽可能多的程序）
  - 2. 重定位方式的确定
    - 决定采用静态重定位或动态重定位，并进行相应的工作

存储器管理的目的和功能

- 存储器管理中的主要功能包括：
  - 3. 实现保护与共享
    - 存储空间保护使各进程不能访问彼此的存储空间，主要方法有：
      - ✓利用处理器的特权级，防止跨级访问
      - ✓利用MMU的界限寄存器规则，防止同级间访问
      - ✓操作系统设置和检查存储区域的保护键与当前进程的键及操作是否匹配
    - 存储空间共享：提供进程间共享数据的途径
  - 4. 实现虚拟存储技术
    - 运行比主存储器容量大的程序
    - 在内存中装入尽可能多的程序

存储器管理方法

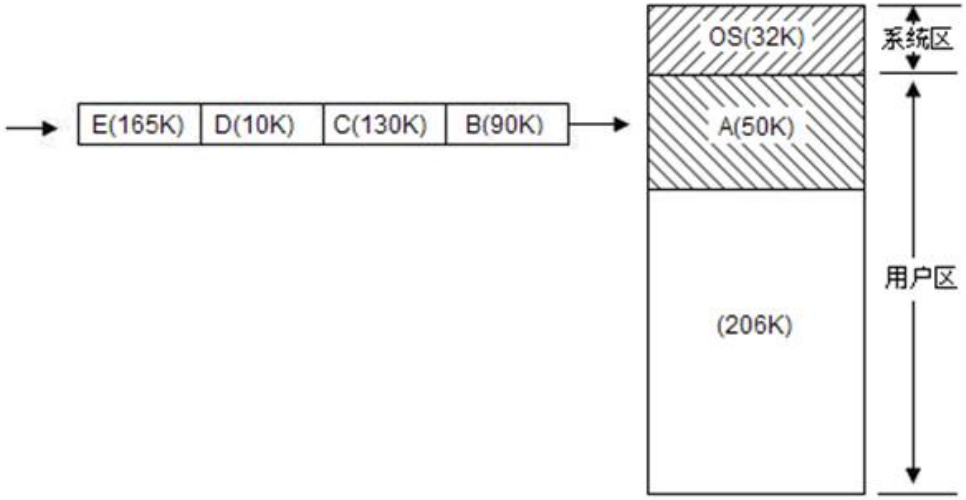
- 存储器管理方法：
  - 分区管理：一个逻辑空间对应物理空间中一个分区，连续分配
  - 分页管理：一个逻辑空间分为多个页，每页装入物理空间的一个块中，可不连续分配
  - 分段管理：进程的数据、代码、堆栈，各自作为独立的逻辑空间
  - 较为先进（复杂）的综合方法：段页式内存管理

单一连续区存储管理

- 单一连续区存储管理：操作系统启动后占用系统区，将整个用户空间看作一个内存区域，一次只能装入一道用户程序，只有用户区中的程序运行完毕方可装入下一道程序。
- 特点：
  - 常用静态重定位
  - 管理简单
  - 无须复杂硬件支持
  - 只允许单任务
  - 可以使用界限寄存器法实现存储保护

单一连续区存储管理

❖ 例：

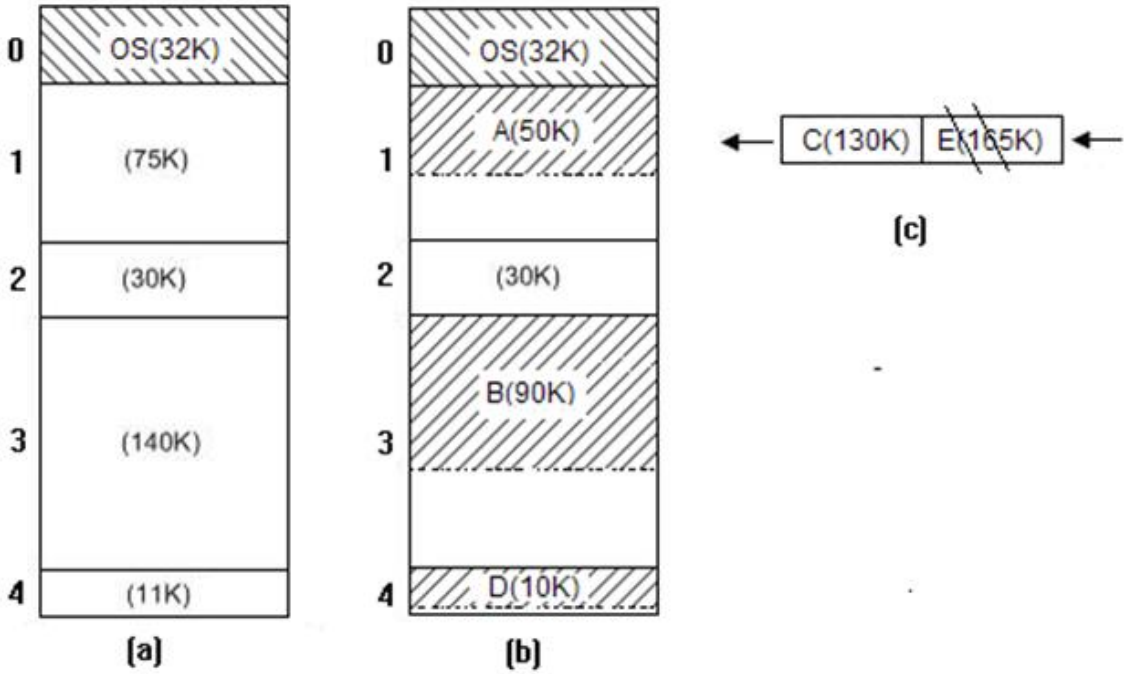


➢ 为实现单一连续区存储管理（以及其他任何存储管理方法），操作系统采用请求表来记录请求装入内存的程序信息。

程序标识	需要的空间长度(虚拟地址空间大小)
A	50K
B	90K
C	130K
D	10K
E	165K

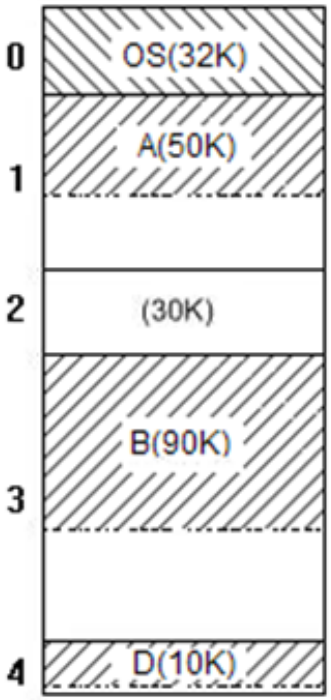
固定分区存储管理

- 固定分区存储管理：操作系统启动后占用系统区，将整个用户区划分为若干个固定大小的区域（大小可不相等）。多道程序分别装入不同的分区内，一道程序只能装入一个分区，而一个分区也只能分配给一道程序。
- 例：



固定分区存储管理

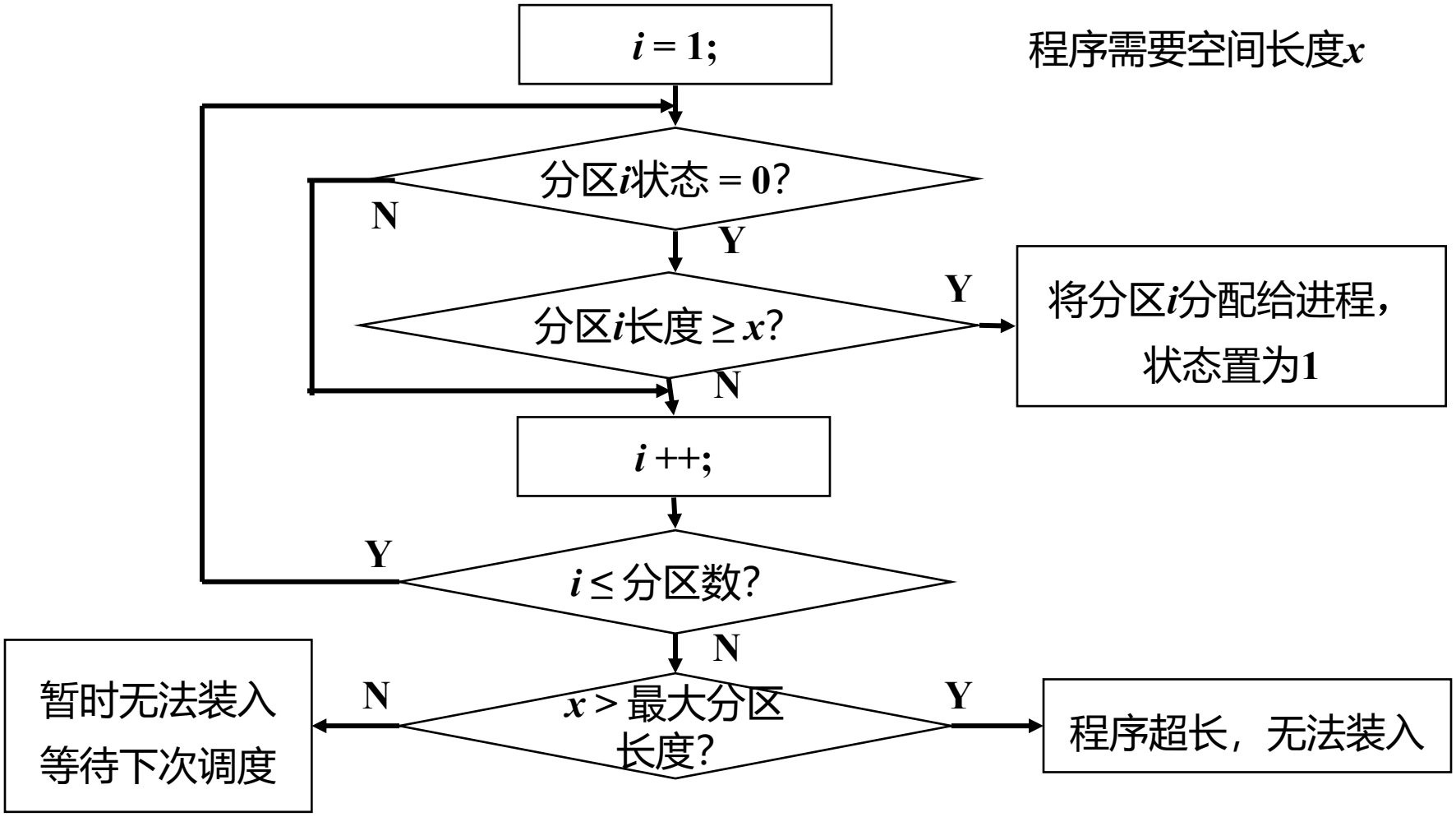
- 实现关键---(1)数据结构设计
  - 为实现固定分区存储管理，操作系统必须记录各分区的位置和使用情况，采用关键数据结构——分区说明表（DPT）



区号	长度	起始地址	状态
1	75K	32K	0
2	30K	107K	0
3	140K	137K	0
4	11K	277K	0

固定分区存储管理

❖ 实现关键---(2)分配和回收

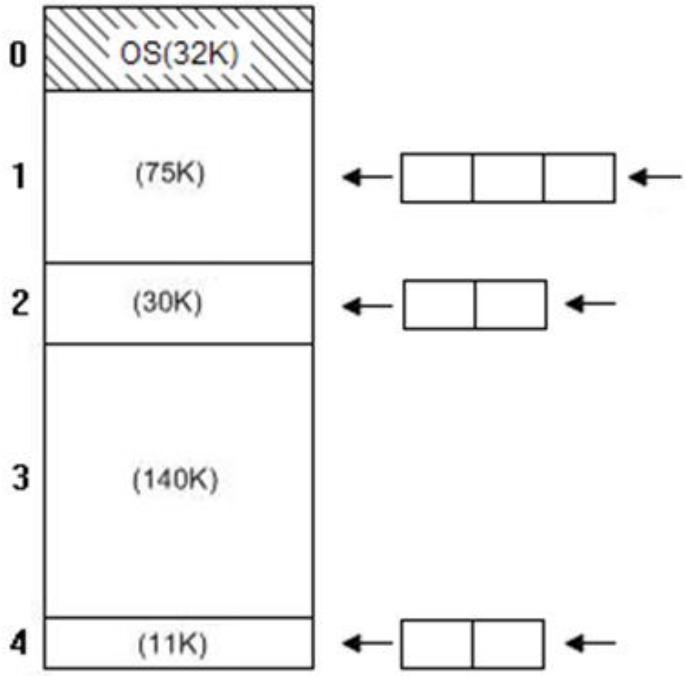


程序需要空间长度x

固定分区存储管理

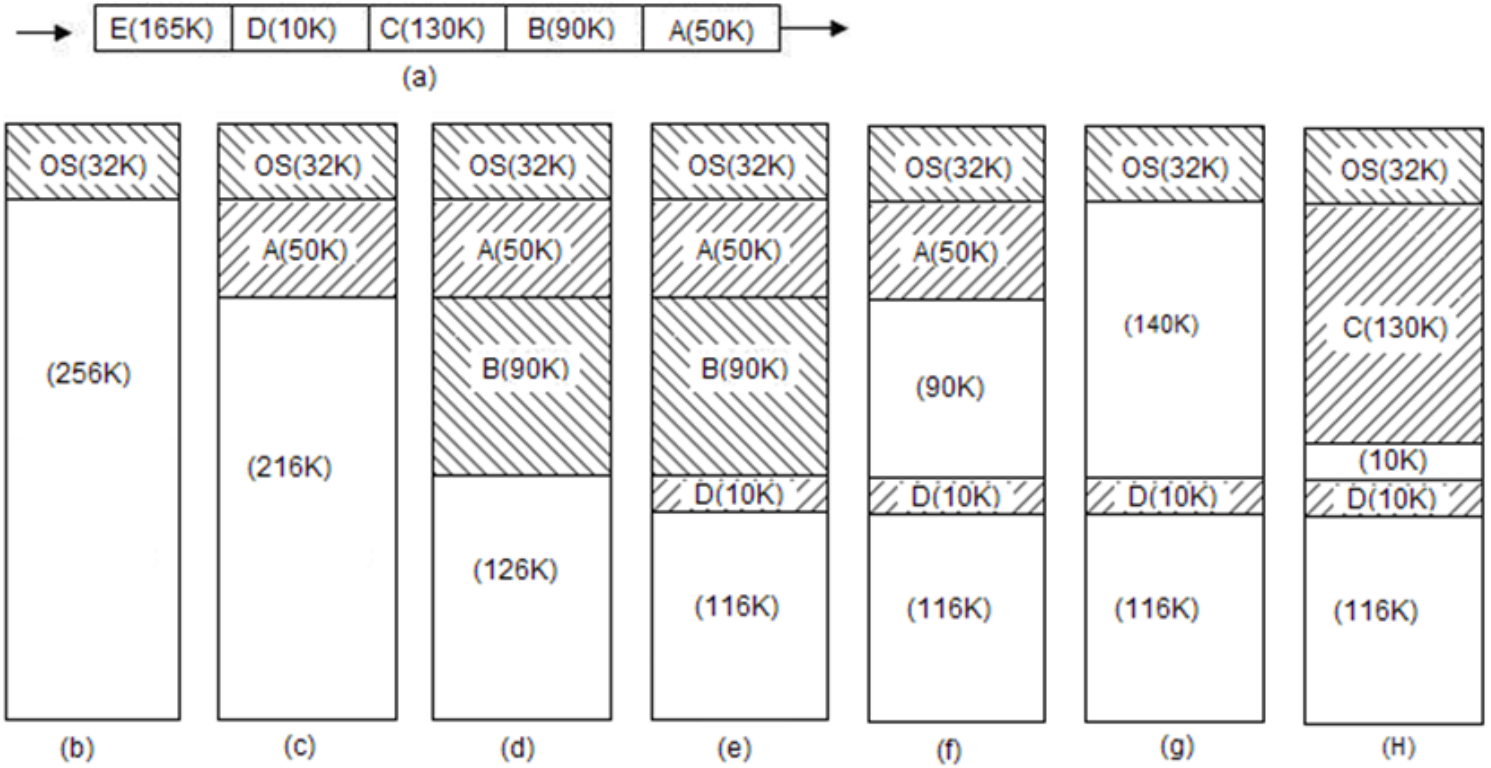
- 实现关键---(3)重定位和存储保护
  - 固定分区存储管理一般采用静态重定位，配合界限寄存器法防止各进程彼此访问存储区域（存储保护）。
- 特点：
  - 支持多道程序设计
  - 并发执行进程数受到分区数限制
  - 程序可用空间受分区大小限制
  - 存储区存在“碎片”

    ——长作业优先、输入队列法



可变分区存储管理

- 可变分区存储管理：在程序装入时选择一个空闲区域并在其中动态创建一个分区以装入程序。
- 例：



可变分区存储管理

- 实现关键---(1)数据结构设计
  - 为实现可变分区存储管理，操作系统需要记录空闲内存区域的位置，可采用以下两种数据结构：
    - 1.可用表
      - ✓类似分区说明表，但其中的记录是变化的，且状态1表示空闲，而状态0表示记录无效（预留）。
      - ✓缺陷：表的长度受到限制，从而导致可以表示的空闲区个数受到限制，影响并发进程的个数。

起始地址	长度	状态
152K	10K	1
172K	116K	1
		0
		0
		0

可变分区存储管理

- 2. 空闲区链表——用链表形式记录空闲区

  ✓链表结点：

```
struct FreeNode {
        long  start;                //分区的起始地址
        long  length;                    //分区的长度
        struct FreeNode *next;    //向下指针 }
```

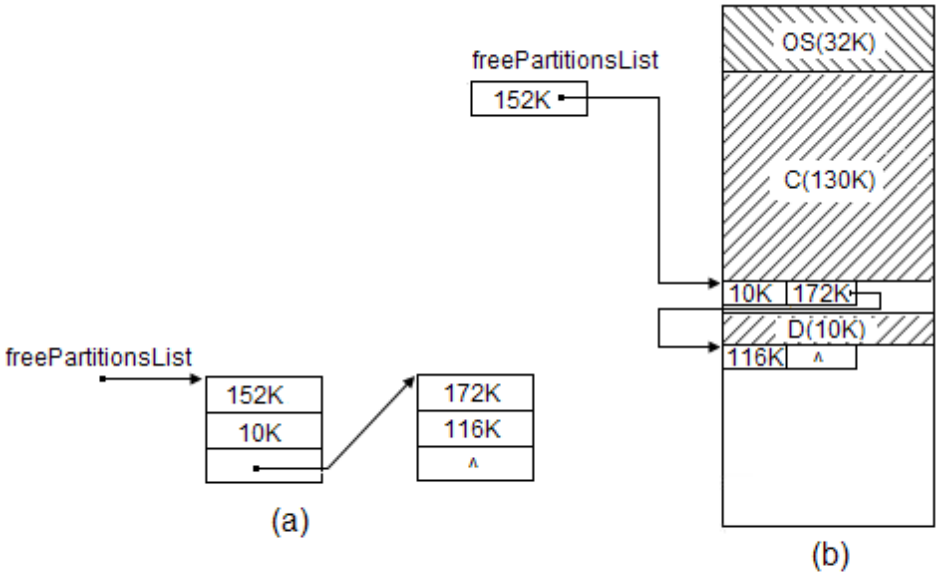


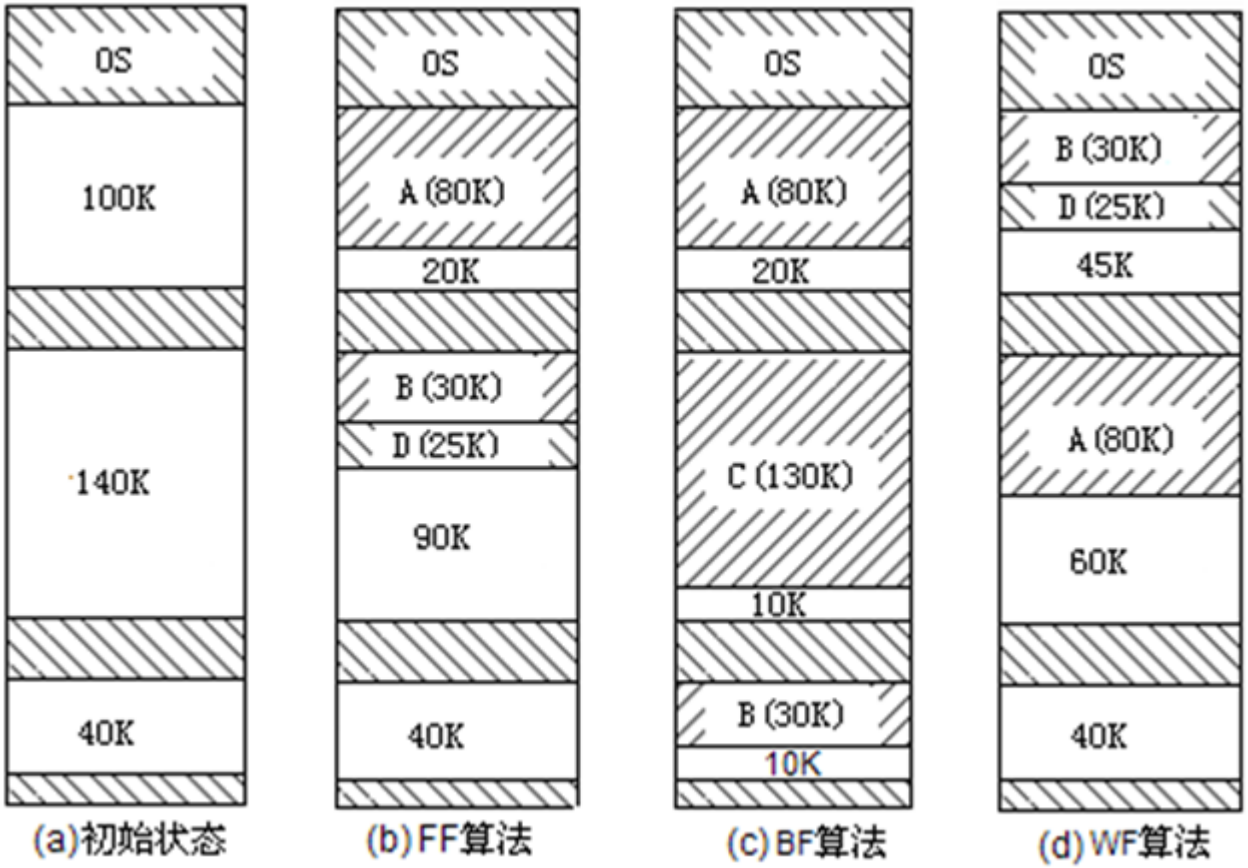
图5-9 空闲区链表的例子

可变分区存储管理

- 实现关键---(2)分配
  - 当程序请求装入时，进行空间分配的算法：
    - 最先适应法（FF）：依次查找空闲区，选择首个足够大的空闲区装入程序
    - 最佳适应法（BF）：在所有足够装入程序的空闲区中选择最小者
    - 最坏适应法（WF）：在所有足够装入程序的空闲区中选择最大者
  - 三种算法的优势和缺点：
    - 最先适应法（FF）：分配速度最快，但对内存使用无规划
    - 最佳适应法（BF）：找到的空闲区与请求最“匹配”，对大程序的装入有充分准备，但易产生（外）碎片
    - 最坏适应法（WF）：不易产生（外）碎片，但容易造成大程序装入的失败

可变分区存储管理

- 例：程序A、B、C和D，它们需要申请的空间大小分别是80K、30K、130K和25K。



可变分区存储管理

- 实现关键---(3)回收
  - 当进行存储区回收时，原先的分配区变为一个新的空闲区，有以下四种情况：



新空闲区前后皆为分配区：增加新空闲区结点



新空闲区前为空闲区，后为分配区：修改前区结点



新空闲区前为分配区，后为空闲区：修改后区结点



新空闲区前后皆为空闲区：修改前区结点，删除后区结点

可变分区存储管理

- 实现关键---(4)重定位及存储保护
  - 为了使可变分区存储管理的空闲区得到更好利用，操作系统需要对（外）碎片进行整合（非必要），这需要将已经装入的程序进行移动，即“移动技术”。因此，可变分区存储管理常采用动态重定位方式。
  - 进程变为运行状态时，其基址和界限将被写入到MMU的基址寄存器和界限寄存器中。
  - 内存访问（重定位）时，MMU首先检查所访问的逻辑地址是否超出界限寄存器中规定的界限值（存储保护），若未超出，则

    物理地址 = 逻辑地址 + 程序基址　　（由于连续装入）

可变分区存储管理

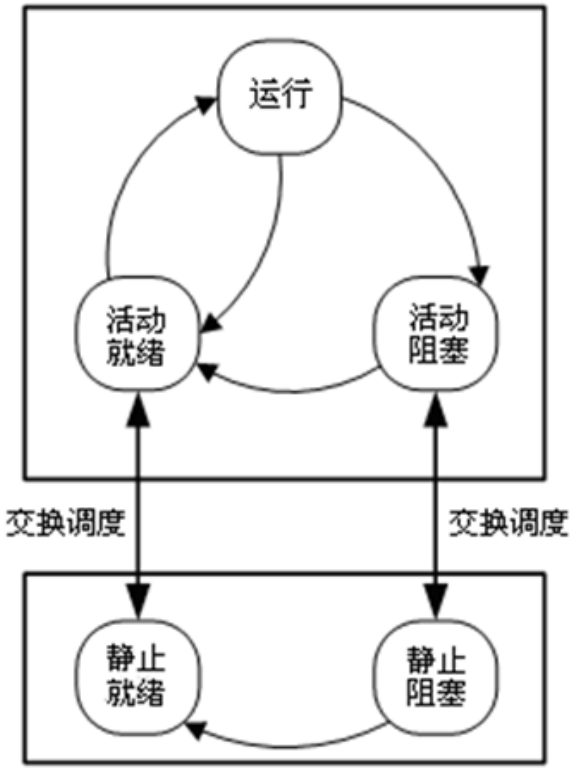
- 可变分区存储管理的特点：
  - 分区个数和每个分区的长度都在变化
  - 存在外碎片(External Fragmentation)，降低了存储空间的利用率
  - 为进程的动态扩充存储空间提供可能
  - 存储区回收时需要相邻空闲区的合并，增加系统的开销
  - 基本分配算法FF、BF和WF，在存储空间利用率上没有很大差别

分区管理方法总结

- 分区管理方法的特点：
  - 程序连续装入，管理（重定位和存储保护）十分方便
  - 存在碎片，存储空间利用率不高
    - 内碎片——单一连续区和固定分区管理中除程序占用外的区内部分
    - 外碎片——动态分区管理中被分配区分割出的小空闲区
  - 程序大小受分区限制
    - 单一连续区和动态分区管理的程序大小不能超过用户区总长度
    - 固定分区管理的程序总大小不能超过最大分区长度

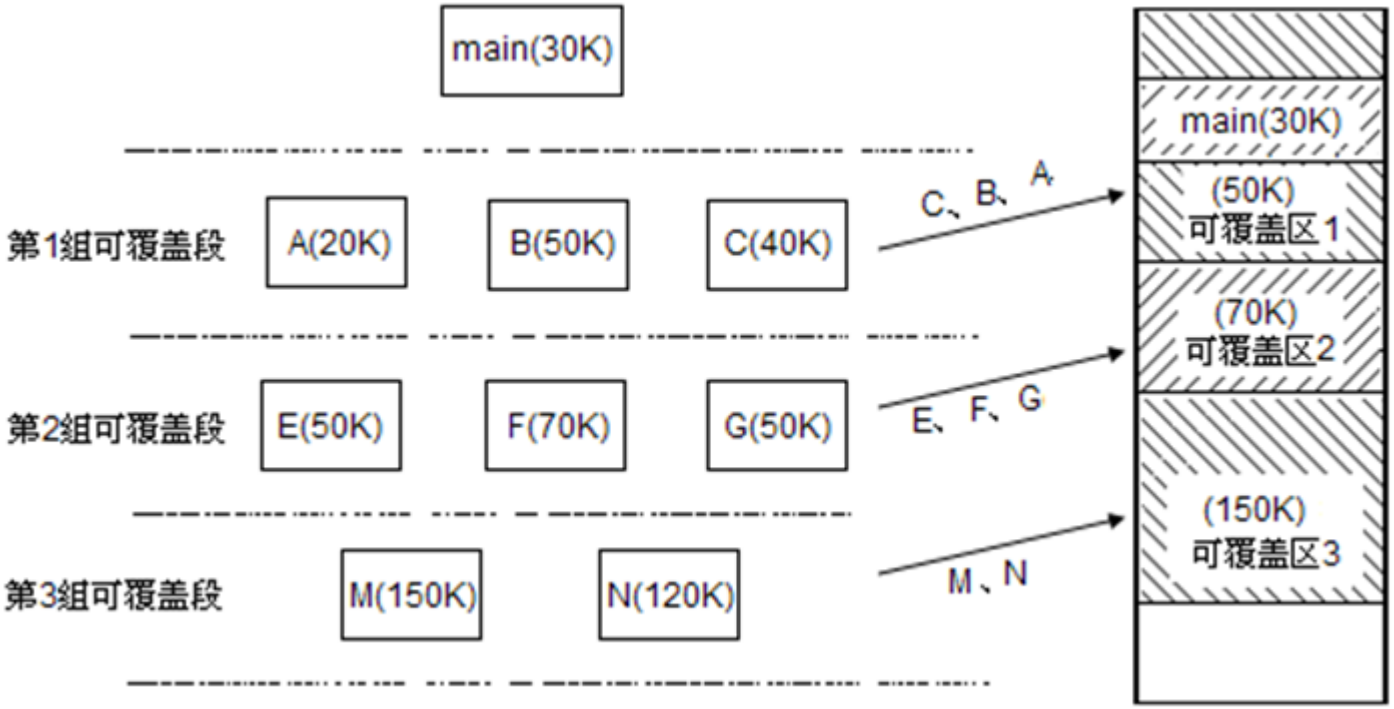
对换技术

- 对换技术的思想：在外存储器中设置一个专门的区域（交换区），将某些就绪或阻塞的进程从内存调出到交换区，内存中空出来的存储区可以用于装入新的进程，或者从交换区中调入之前调出的进程，或者供当前运行的进程扩充存储空间。
  - 一种内存逻辑扩充技术
  - 增加并发执行的进程数量
  - 让当前运行的进程拥有更多的可用存储空间
  - 减少就绪队列进程数，从而减少调度时间
  - 增加进程调度的灵活性



覆盖技术

- 覆盖技术是另一种对内存进行逻辑扩充的技术。它允许程序自主控制将其中的子程序装入内存的可覆盖区，并自主控制其执行。



对换与覆盖

- 对换技术与覆盖技术的比较：

	对换技术	覆盖技术
控制者	操作系统	用户程序
粒度	整个进程	子程序
单个程序空间	仍然受限	可能突破限制
并发执行程序数	不受限制	仍然受限

分区-作业1

例1.1：已知某分时系统有一组依次同时到达的请求(如下表所示)，系统使用可变分区存储管理方法（未使用移动技术），用户区长度为256K。假定当前系统时间是0，程序装入I/O操作与CPU可以并行执行，忽略调度等的系统开销。

(1)作业调度采用短作业优先算法，进程调度采用简单RR算法，时间片长度为5。画出进程调度图，并给出各请求的周转时间，计算平均周转时间。

请求	虚拟地址空间大小	所需CPU时间
A	120K	8
B	90K	4
C	180K	6
D	30K	3

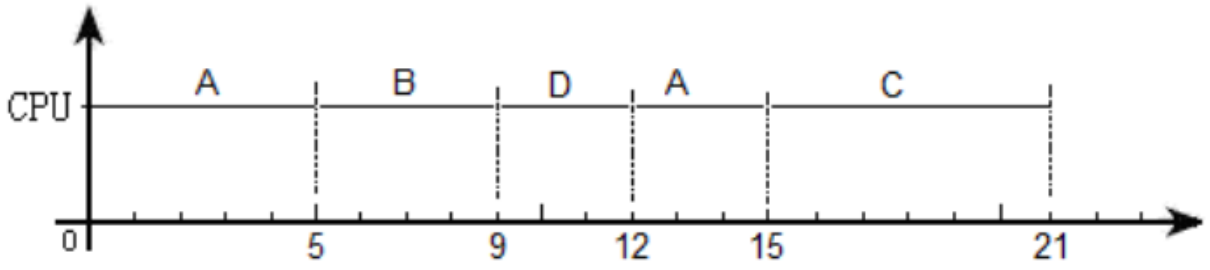
分区-作业2

例1.2：已知某分时系统有一组依次同时到达的请求(如下表所示)，系统使用可变分区存储管理方法（未使用移动技术），用户区长度为256K。假定当前系统时间是0，程序装入I/O操作与CPU可以并行执行，忽略调度等的系统开销。完成：

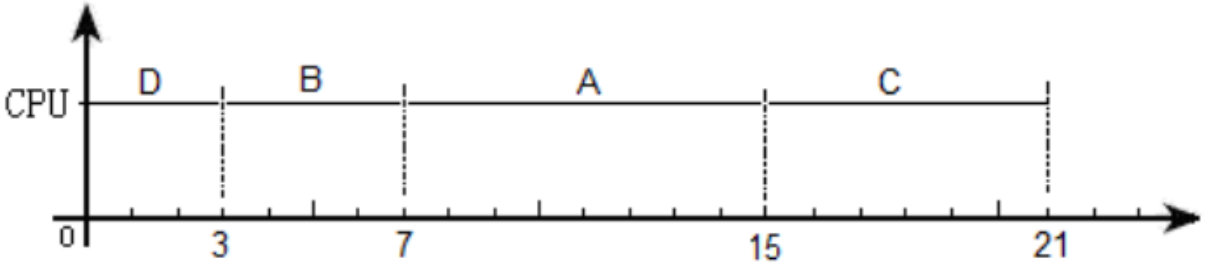
(1)进程调度为简单RR算法，时间片长度为5。画出进程调度图，并给出各请求的周转时间，计算平均周转时间。

(2)进程调度为短进程优先算法，画出进程调度图，并给出各请求的周转时间，计算平均周转时间。

请求	虚拟地址空间大小	所需CPU时间
A	120K	8
B	90K	4
C	180K	6
D	30K	3



TA=15, TB=9, TC=21, TD=12　T=(15+9+21+12)/4=14.25　（5 分）



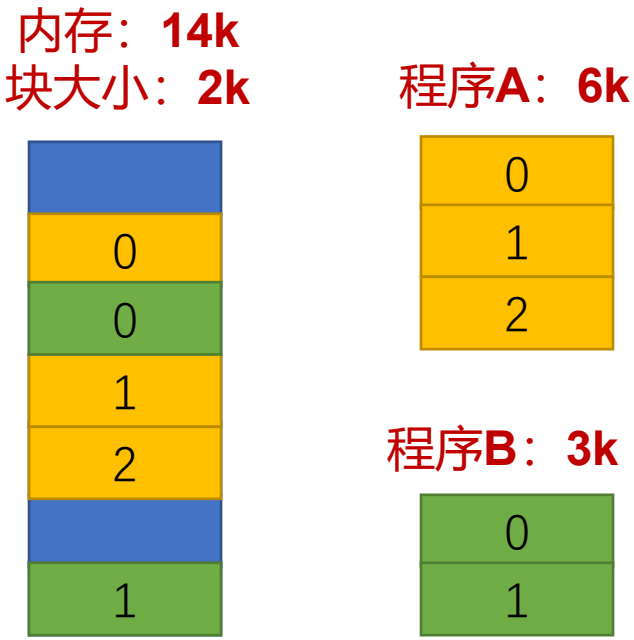
TA=15, TB=7, TC=21, TD=3　T=(15+7+21+3)/4=11.5　（5 分）

计算机操作系统

第5章 存储器管理(2)

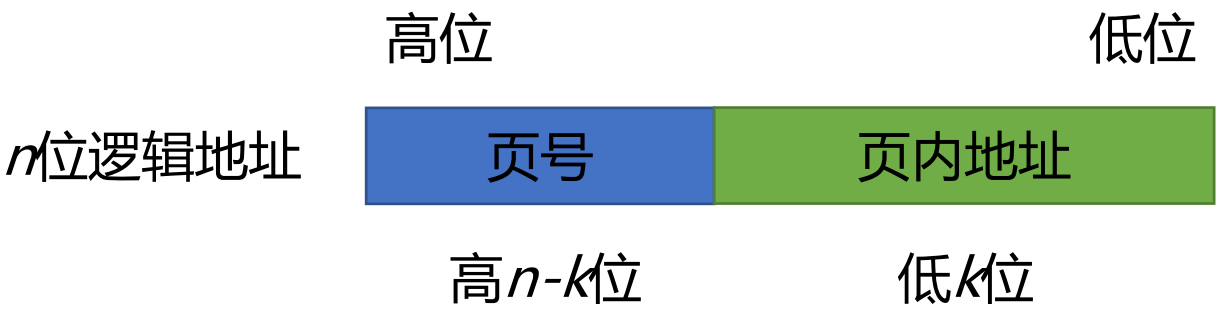
分页存储管理

- 内存分块：将内存空间分割为若干个等长的小区域，这些区域称作块或帧。块的大小通常为2^k字节。
- 程序分页：程序装入内存前按照块的大小分为若干页，依次将这些页编号为0、1、2、……
- 程序的装入：每一页被装入到内存的一块中，块（页）内连续装入，但块的位置可以是不连续的。



分页存储管理

- *n*位逻辑地址的高位和低位分别指示了其页号和页内地址



- 分页机制下，若块（页）的大小通常为2^k字节，则某逻辑地址*a*对应的页号*p*和页内地址ω:

$$p = a / 2^k \qquad p = a >> k$$

$$\omega = a \,\%\, 2^k \qquad \omega = a \,\&\, (2^k - 1)$$

分页存储管理

- 根据页被装入的时机，分页存储管理分为两类：
  - 静态（基本）分页：程序装入内存时装入其所有的页
    - 运行速度快，实现方法简单
    - 程序所需空间不能超过实际内存大小
  - 动态分页：程序装入时仅装入某些页，而其他的页暂时保存在外存中，待需要使用时再替换内存中的页
    - 程序所需空间能够超过实际内存大小
    - 运行速度慢，实现方法复杂

静态分页的实现关键—数据结构的设计

❖ 1. 记录内存中各块的使用（空闲/被分配）情况

➢ 位示图法：内存的第*b*个块的使用情况被记录在一个位示图第*i*个字的第*j*位上，0表示空闲，1表示已被分配

$$b = 字长 \times i + j$$

$$i = b \,/\, 字长$$

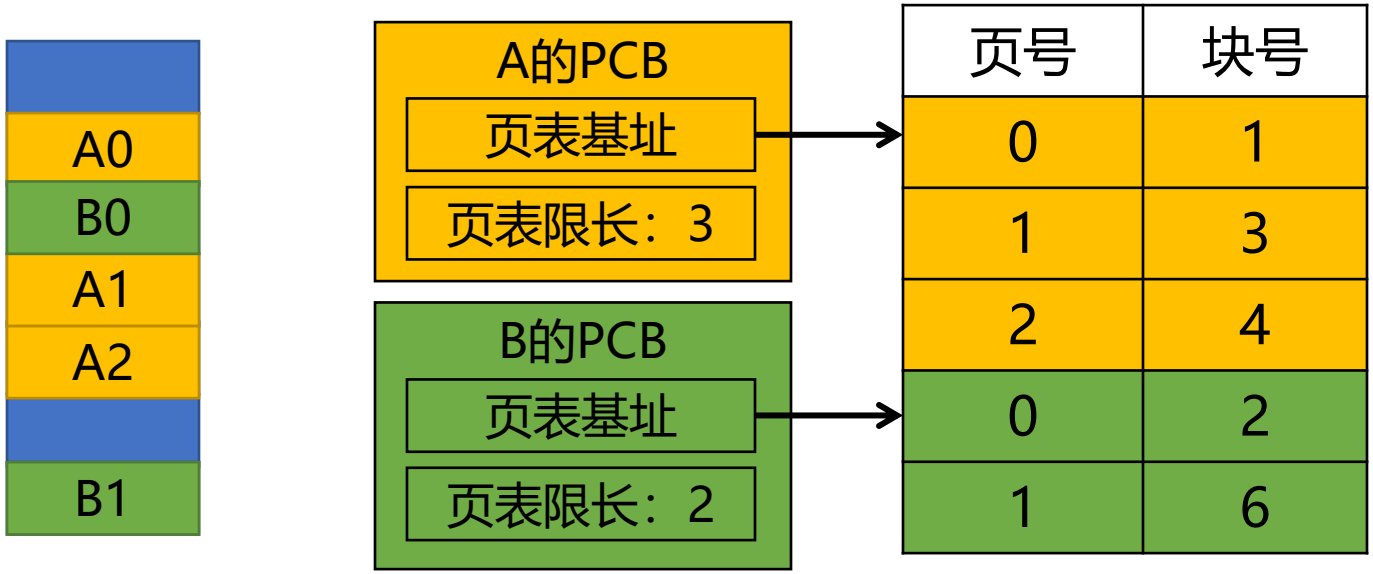
$$j = b \,\%\, 字长$$

*n*个字的位示图可表示“字长×*n*”个块。

➢ 空闲块链表法：建立链表来记录空闲块的编号。

静态分页的实现关键—数据结构的设计

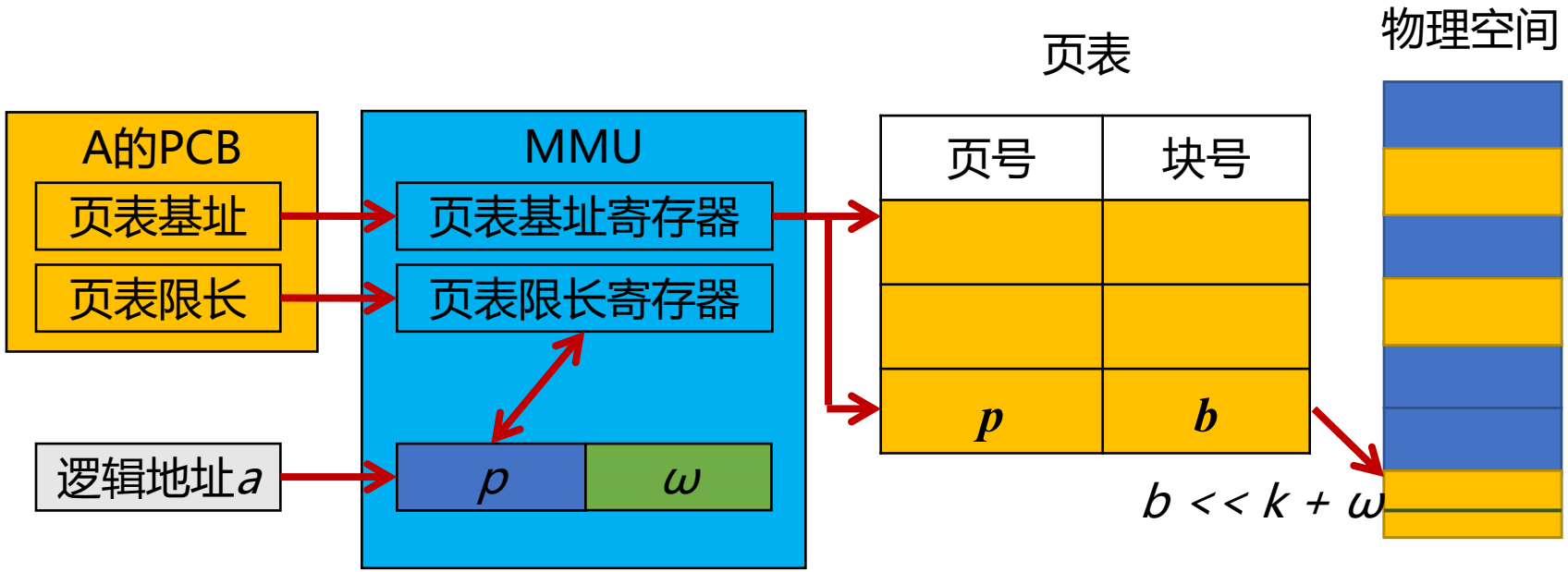
- 2. 记录进程中的页与内存中的块的对应情况
  - 页表：每个进程创建时将建立该进程的页表，以记录进程的各个页被装入到内存中的块的编号。操作系统还将页表的起始位置（页表基址）及页的个数（页表限长）记录在进程的PCB中以备重定位时MMU使用。



静态分页的实现关键—重定位及存储保护

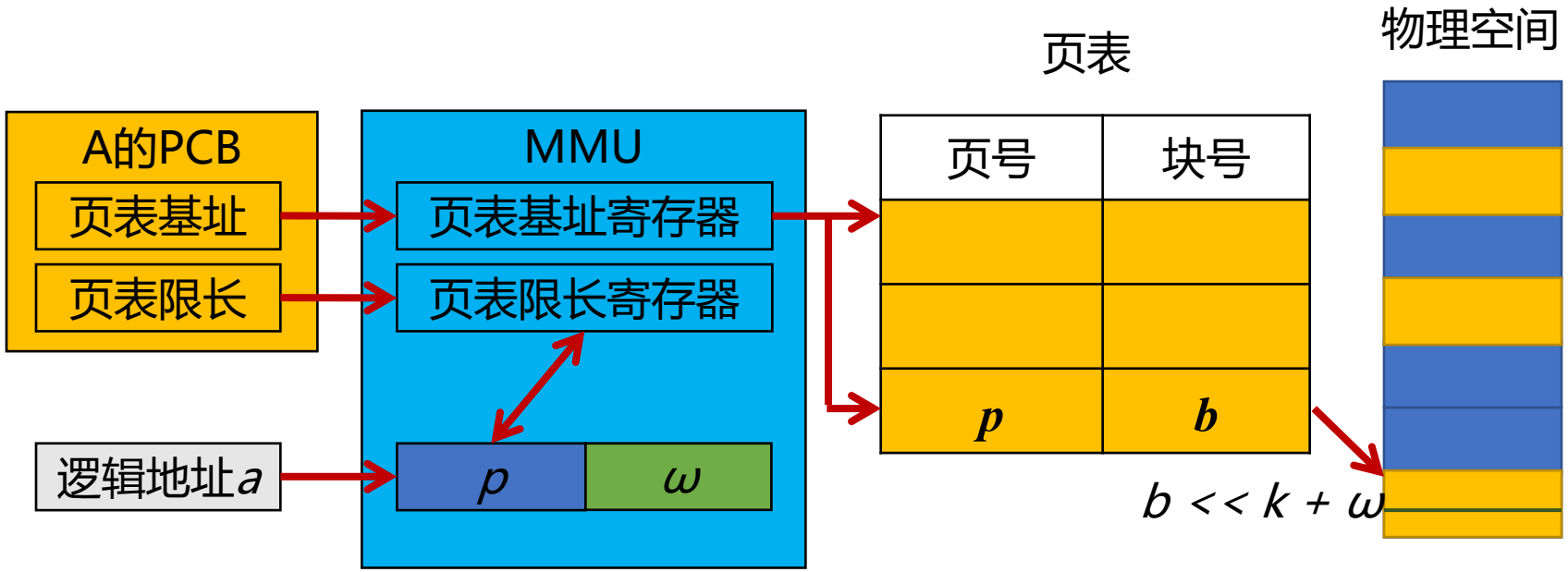
- 静态分页技术采用动态重定位，同时进行存储保护，实现过程：

进程调度程序将A的PCB中的页表基址和页表限长装入MMU的寄存器中。



静态分页的实现关键—重定位及存储保护

- 静态分页技术采用动态重定位，同时进行存储保护，实现过程：
  - 进程创建时，建立页表，记录页表基址、页表限长。
  - 进程变为运行状态时，将页表基址和页表限长写入MMU的寄存器中。
  - 进行内存访问时，MMU区分逻辑地址的页号与页内地址
  - MMU检查页号是否越界（超出页表限长）
  - MMU按照页号查找页表以确定块号
  - MMU根据块号与页（块）内地址确定物理地址



静态分页的实现关键—重定位及存储保护

- 静态分页技术采用动态重定位，同时进行存储保护，实现过程：

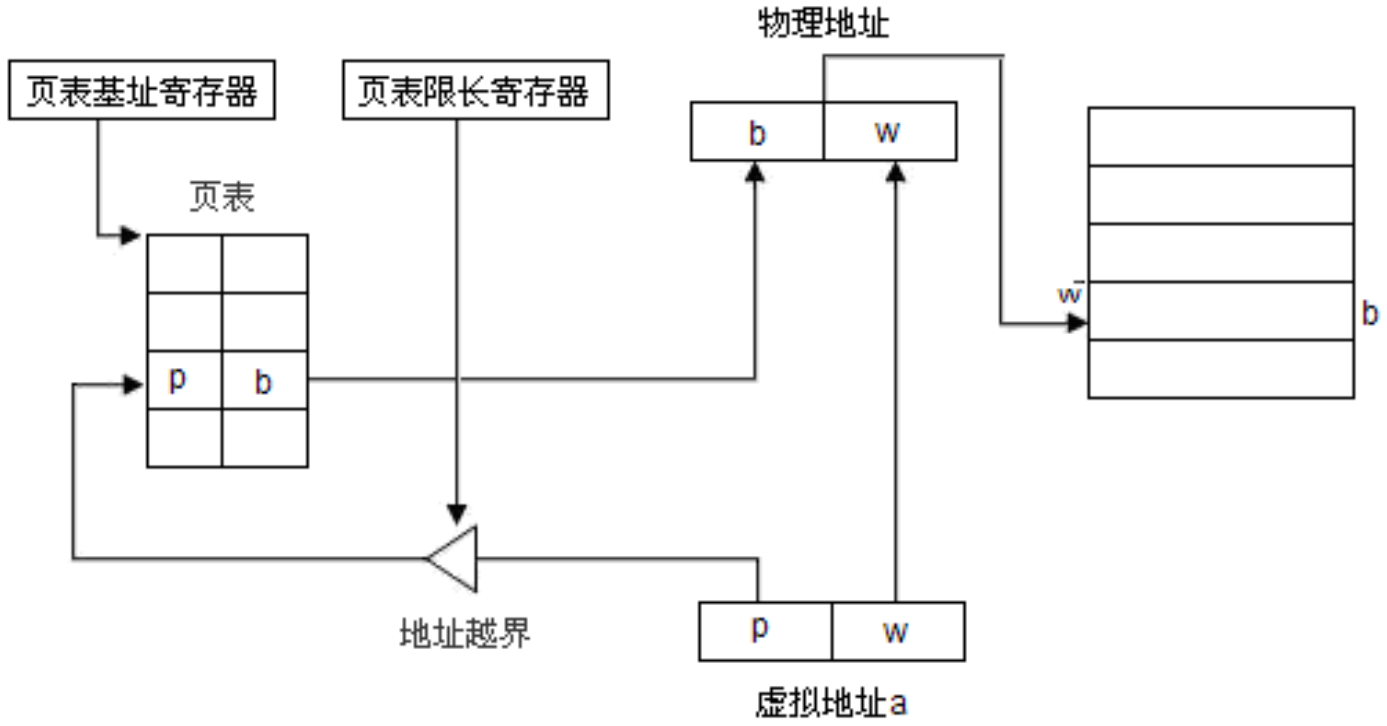


图5-21 分页的重定位

静态分页的实现关键

- 例：在某静态分页存储管理中，已知内存共32块，块长度为4K，当前位示图如下。现有一进程P正在创建，其逻辑地址空间大小为50K。问：
  - (1) 进程P共有几页?
  - (2) 给出进程P的页表。
  - (3) 进程P的访问逻辑地址： 8192(十进制)和0x5D8F(十六进制)，计算对应的物理地址。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	1	0	1	1	0	1	1	1	1	1
0	0	0	0	0	1	1	0	0	0	1	0	0	1	1	0
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16

静态分页的特点及效率的改进

- 优点：
  - 允许非连续存储，提高存储器利用率
  - 页长度与块长度相等，无须采用复杂分配策略
  - 为虚拟存储技术提供基本数据结构支持
- 不足及解决方案：
  - 重定位时两次访存（查找页表和读取数据），增大了CPU的开销。
  - 解决方案：利用特殊存储硬件——“快表”（TLB：Translation Lookaside Buffers）存储进程的部分页表，访存时先在TLB中查询。
  - 给定时间段内，MMU在访问页对应的块时，若有m次在快表中得到块号，有n次在页表中得到块号，那么，这个时间段MMU的快表命中率：

$$h = \frac{m}{m+n} \times 100\%$$

静态分页的特点及效率的改进

- 具有快表的静态分页重定位：

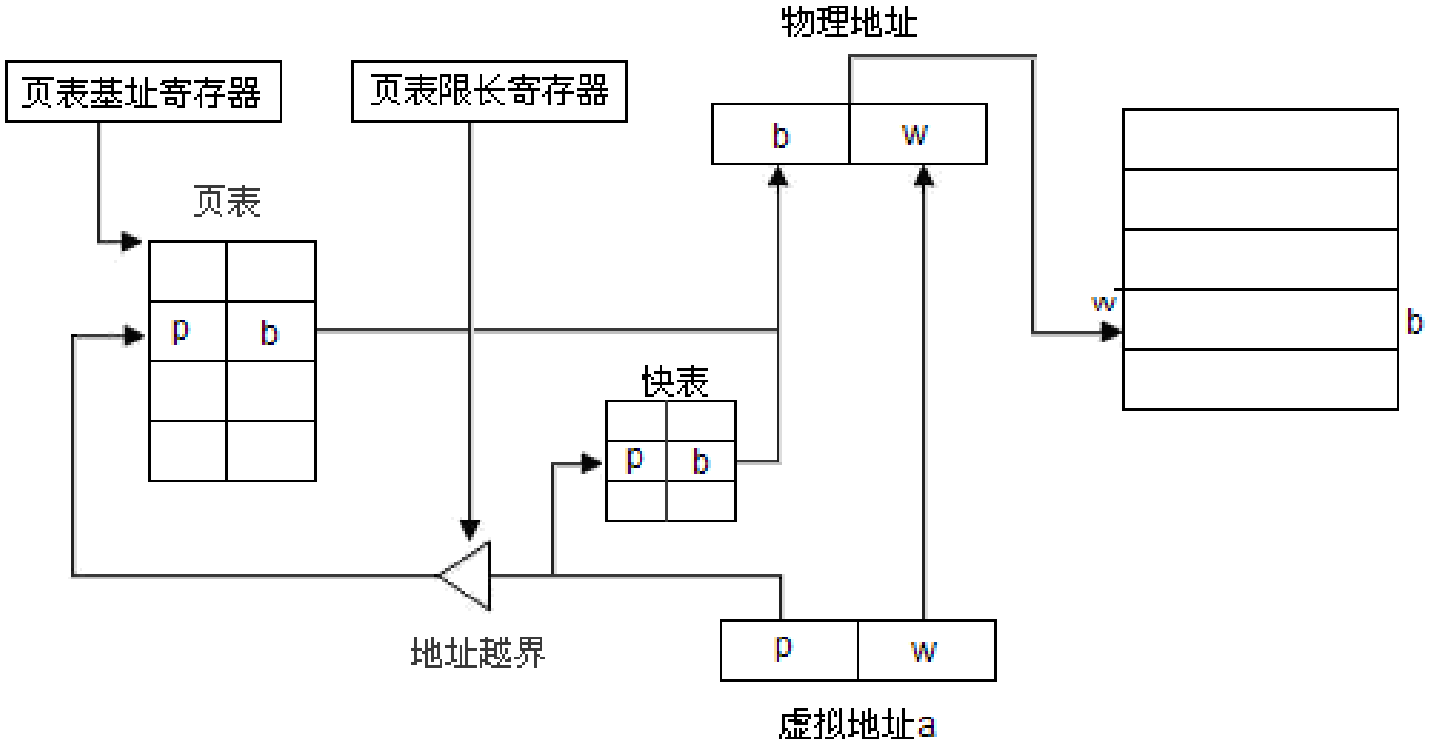


图5-24 具有快表的重定位

虚拟存储与动态分页

- 虚拟存储思想
  - 程序装入时，只装入基本的部分信息，其余部分仍然保留在外存中；
  - 程序运行过程中，根据需要动态地把程序的其他部分调入或者把已调入的部分调出内存。
- 虚拟存储目的
  - 使系统仿佛拥有充分大的内存，从而可以运行超过其物理内存容量的大程序，或者在主存中装入尽可能多的程序。
- 虚拟存储要解决的主要技术
  - 理论基础（程序的局部性原理）
  - 调入策略
  - 置换算法

虚拟存储与动态分页

- 程序的局部性原理：
  - 在程序运行过程的一个较小时间范围内，只需要一小部分的程序信息，其他部分暂时不需要。
  - 在程序的一次执行过程中，程序的所有指令和数据被访问的概率间存在不平衡性。
- 程序的局部性原理分为：
  - 时间局部性：某地址的信息被访问后，短时间内很可能再次被访问。
  - 空间局部性：某地址的信息被访问后，与其临近的信息很可能被访问。

虚拟存储与动态分页

- 动态分页技术按页对程序进行动态的调入调出，是实现虚拟存储的理想方法之一。
  - 动态分页技术按照虚拟存储的调入策略分为：
    - 请求分页（请求调入）：当处理器发现需要访问的信息不在内存中时，将其调入内存。
    - 预调入分页：在处理器访问信息前，先将信息调入内存，保证处理器需要访问的信息总已在内存中。
  - 调出策略---置换算法

请求分页的实现关键—扩展页表

- 与静态分页技术不同，在动态分页中，页表不但需要记录页对应的块的编号，还需记录：
  - 页是否在内存中？ 中断位P
  - 页在装入后是否再次被访问过？ 访问位A
  - 页在装入后是否被修改？ 修改位M
  - 页在外存中的位置 外存地址

请求分页的实现关键—重定位

- 请求分页的重定位过程与静态分页大致相同（由MMU完成）：
  - 根据页号在快表中查找，若找到且通过访问权限检查则访问物理地址，置该页A=1。
  - 若在快表中未找到且页号未越界，则在页表中查找，若找到的页P=1且通过访问权限检查则访问物理地址，置该页A=1（并将此页信息写入快表中）。
  - 若页表中找到的页P=0，则MMU发出一个缺页中断。

请求分页的实现关键—缺页中断

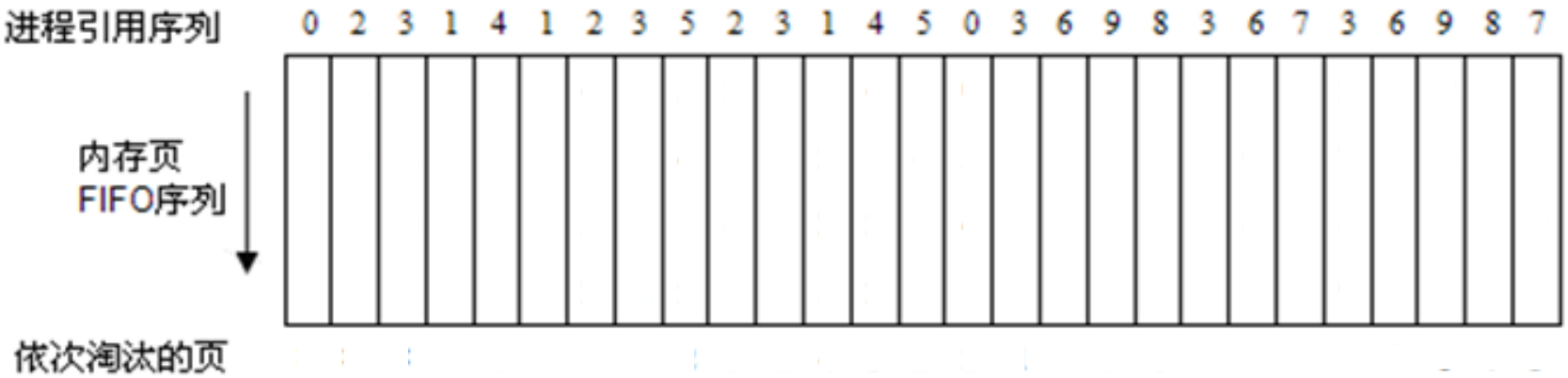
- 当缺页中断发生时，将进入操作系统的缺页中断处理程序：
  - 保护现场
  - 获取空闲块
    - 若系统有空闲块，则选择一个空闲块
    - 若无空闲块，则根据某种策略，选择一个页，保存其已发生的修改（判断M位），并调出内存（P=0），以得到一个空闲块
  - 将所缺的页写入空闲块中，修改页表信息，例如：块号写入页表，置P=1，A=0，M=0

请求分页的实现关键—页面调度和置换算法

- 当需要选择一个淘汰页时，可采取两种页面调度方式之一：
  - 局部页面调度——只在当前运行进程的页面中选择淘汰页
  - 全局页面调度——可以在所有进程的页面中选择淘汰页
- 置换（淘汰）算法是选择淘汰页的具体策略。
  - 最理想的置换算法能选择今后不会被访问到或者最长时间之后才会被访问到的页来作为淘汰页。但这一目标难以实现。
  - 好的置换算法可以减少缺页中断发生的次数，避免反复进行调入调出（抖动现象）。

请求分页的实现关键—置换算法

- 1. 先进先出（FIFO）算法
  - 选择存在于内存中最久的页作为淘汰页
  - 例子 一个进程运行过程依次访问的页号(也称进程的引用序列)是：0、2、3、1、4、1、2、3、5、2、3、1、4、5、0、3、6、9、8、3、6、7、3、6、9、8、7。假定该进程可以使用4个块，按局部页面调度，采用FIFO算法时，如何计算缺页中断的次数？依次淘汰的页号是哪些?

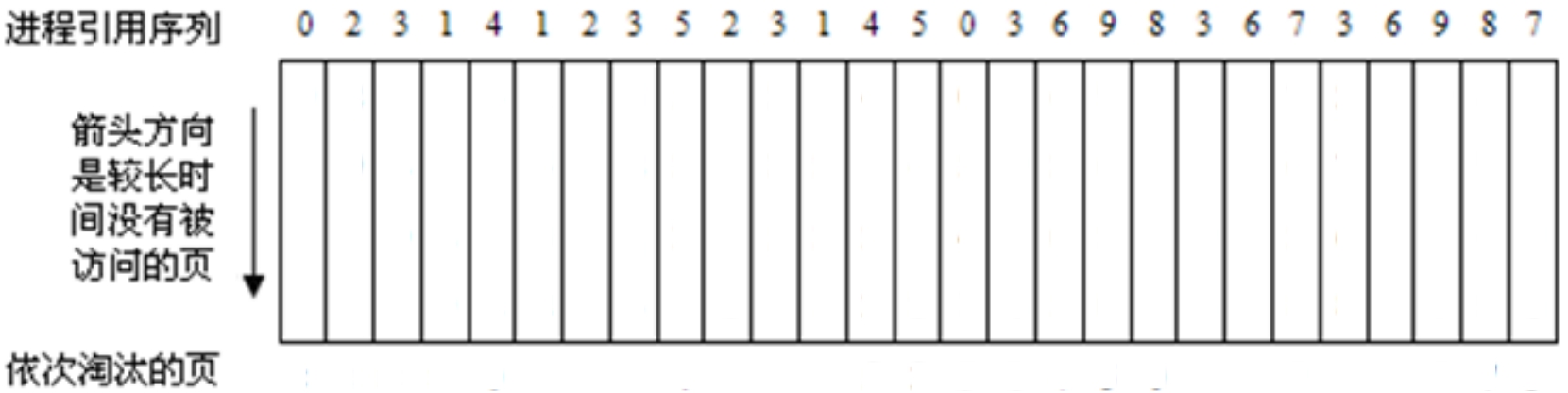


  - FIFIO算法算法简单、容易实现，但是可能会将经常访问的页淘汰。

请求分页的实现关键—置换算法

❖ 2. 最久未使用（LRU）算法

➢ 选择最长时间未被使用的页作为淘汰页

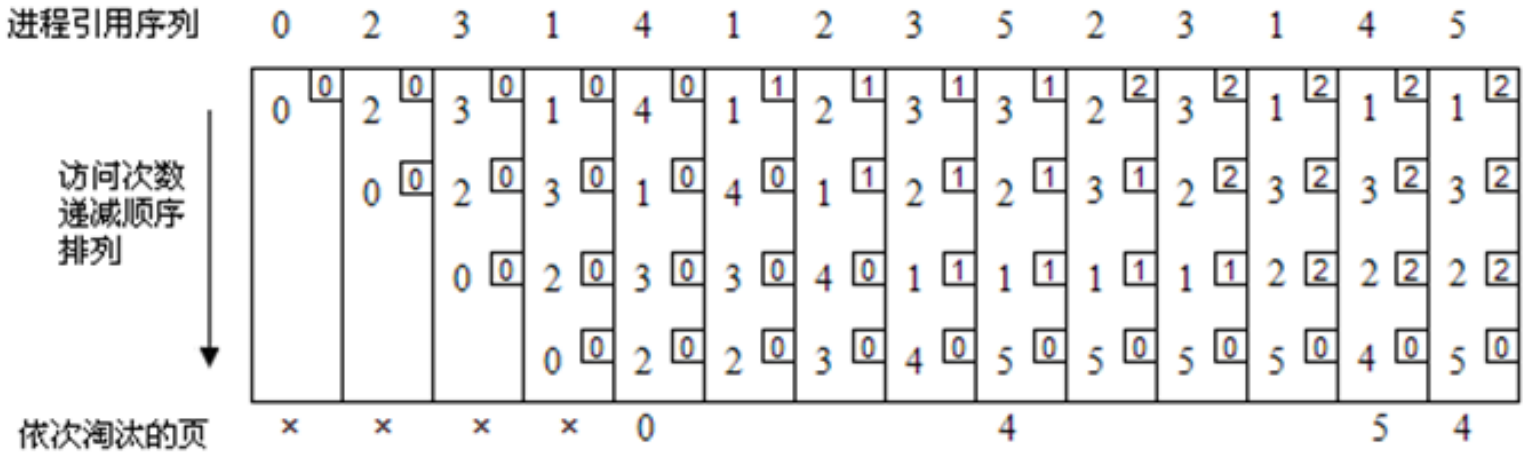


➢ LRU算法在实际应用中取得很好效果，但需要硬件特殊支持（重定位未缺页时修改页序列）。

请求分页的实现关键—置换算法

❖ 3. 最不常用（LFU）算法

➢ 记录页面使用次数，选择最少被使用的页作为淘汰页



➢ LFU算法也需要硬件特殊支持（重定位未缺页时更新页的使用次数记录），且使得新装入的页很容易被淘汰。

请求分页的实现关键—置换算法

- 4. 最近未使用（NRU）算法——LFU或LRU的近似实现
  - 定期重置A=0，选择最近（上次重置以来）未被使用的某页

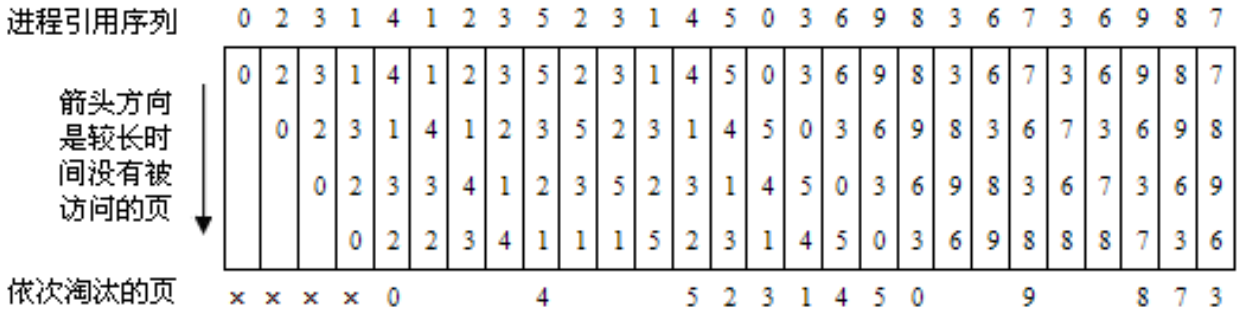


图5-27 LRU置换算法的例子

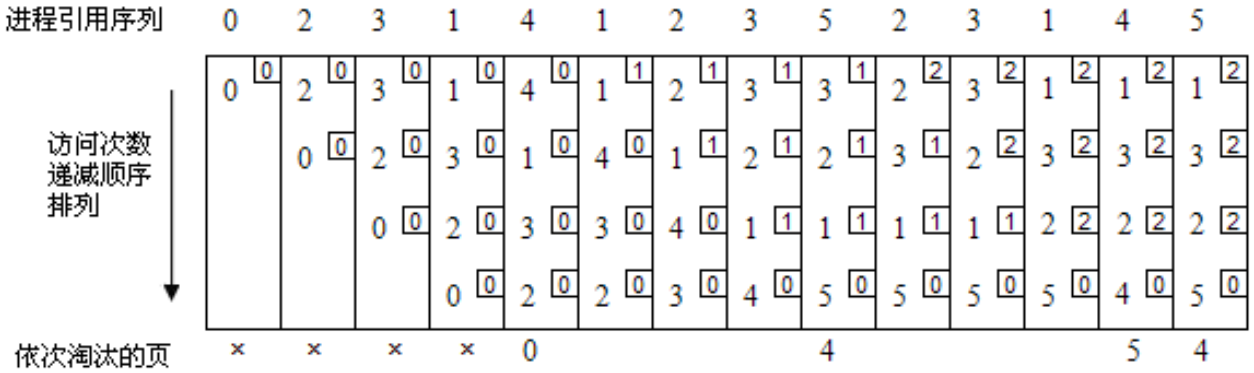
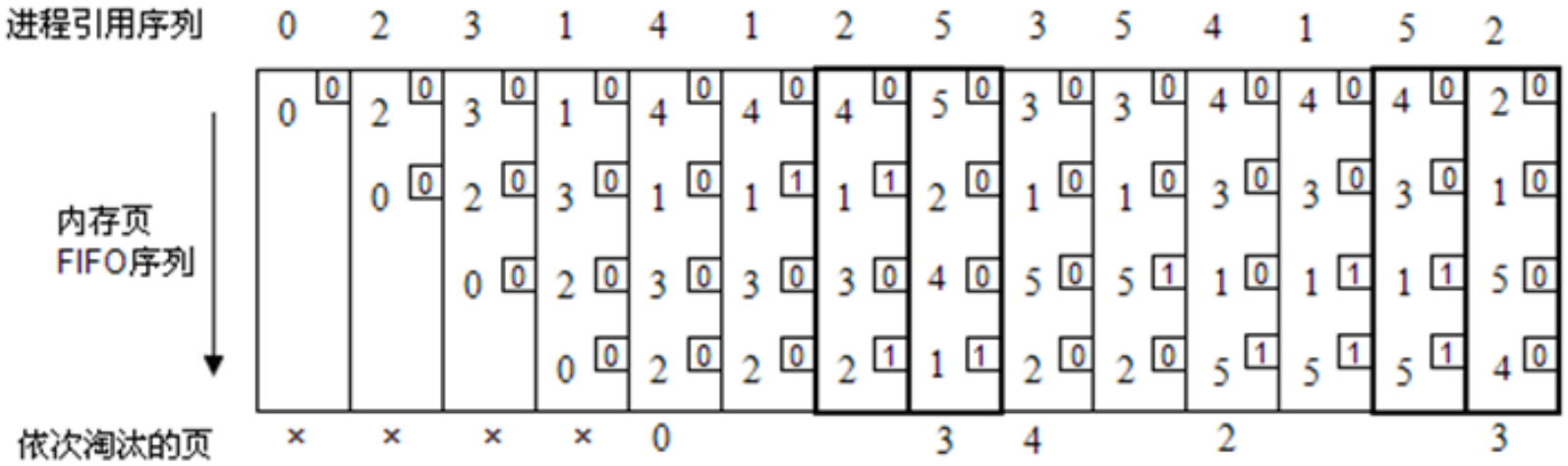


图5-28 LFU置换算法的例子

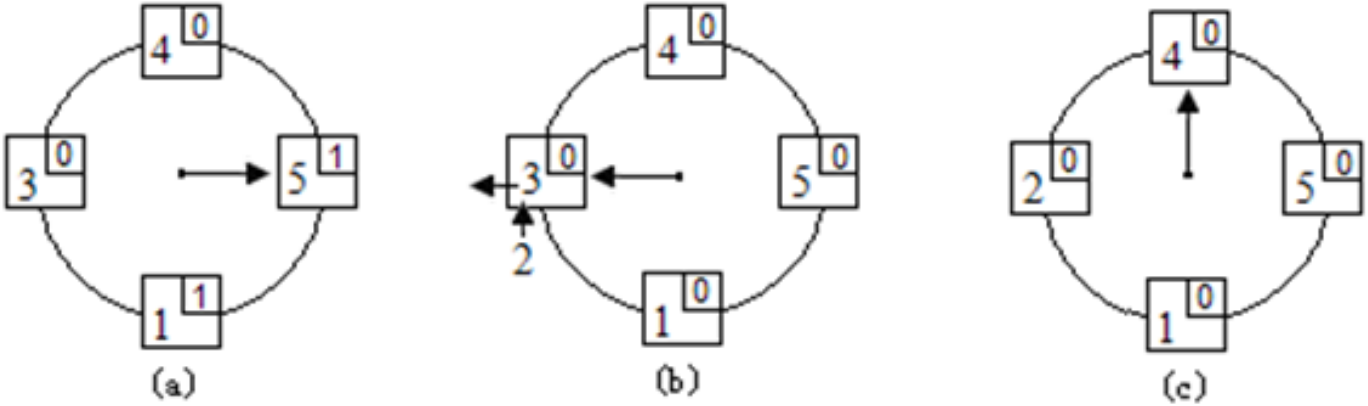
请求分页的实现关键—置换算法

- 5. 二次机会（Second Chance）算法
  - 在FIFO的基础上考虑了页的使用情况。具体：在FIFO的基础上选择淘汰页，但淘汰前进行检查，若选择到的页A=1，则不淘汰此页，重置其A=0并将其放置回页序列末端，并转而选择下一页并检查。



请求分页的实现关键—置换算法

- 5. 二次机会（Second Chance）算法
  - 二次机会（Second Chance）算法也被称为时钟算法，因为页序列可以放置在一个钟面上来考虑。



请求分页的实现关键—置换算法

❖ 5. 改进型二次机会算法

➢ 二次机会算法还可以同时考虑保存页（若被修改）的开销。

- 将页分为四类：
  - ✓① A=0，M=0。淘汰这类页I/O开销小，抖动现象可能性小；
  - ✓② A=0，M=1。淘汰这类页I/O开销大，抖动现象可能性小；
  - ✓③ A=1，M=0。淘汰这类页I/O开销小，抖动现象可能性大；
  - ✓④ A=1，M=1。淘汰这类页I/O开销大，抖动现象可能性大；
- 检查步骤：
  - ✓(1) 查找①类页的，如果找到则淘汰，算法完成；
  - ✓(2) 查找②类页的，如果找到则淘汰，算法完成；查找过程中，置A=0.
  - ✓(3) 返回(1)，进行第二遍查找

请求分页的实现关键—置换算法

- 6. 页缓冲算法
  - 设置两个容量比例阈值T0和T1，其中T0 > T1。
  - 在某次装入缺页的过程中
    - 若空闲区比例“越过”T0，则重置所有页的A=0
    - 若空闲区比例“越过”T1，则将所有A=0的页信息存入链表M0（未被修改的页）和M1（被修改的页）中，并将这些页“淘汰”（仅置P=0，但未被替换）
  - 当缺页发生时
    - 若所缺页在M0或M1中，则恢复其P=1，并移除出链表
    - 若所缺页不在M0或M1中，则获取空闲块并装入缺页，其间先尝试淘汰M0中的页，若M0为空，则淘汰M1中的页，或者所有页

请求分页的实现关键—工作集模型

- 一个进程当前正在使用的页的集合称作当前的“工作集”。
- 若进程的当前工作集全部存在于内存中，则在一段时间内不会发生“缺页”，从而避免抖动。
- 一些操作系统设置专门程序来检测和预测进程的工作集，以便：
  - 为置换（淘汰）算法做进一步参考
  - 当总工作集大小超出内存块总量时，考虑使用对换技术

请求分页的实现关键—缺页率

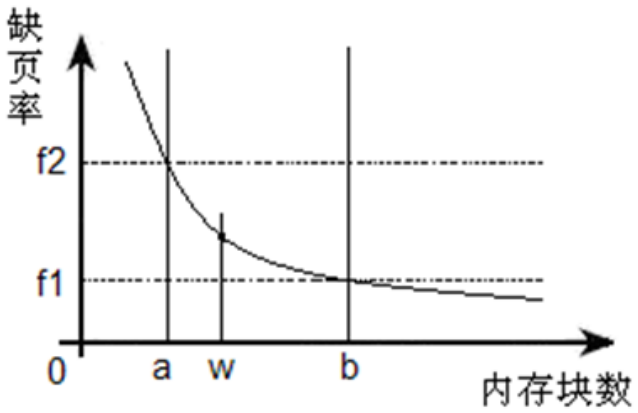
- 缺页率是分页管理的评价指标之一。

如果一个进程在一段时间内的引用序列长度为n，它在执行过程中，产生缺页中断的次数为m，那么，该进程执行这个引用序列的缺页率定义为：

$$f = \frac{m}{n} \times 100\%$$

一般而言，对于某个进程，可使用的内存块数越多，它在运行过程产生的缺页率越小。

但对于某些置换算法（如FIFO），有可能在一些情况下存在相反的现象，即Belady现象。



补充

3．从何处调入页面

请求分页系统中的外存分为两部分：用于存放文件的文件区和用于存放对换页面的对换区。对换区通常采用连续分配方式，而文件区采用离散分配方式，因此对换区的磁盘 I/O 速度比文件区的更快。这样，从何处调入页面就存在三种情况：

1）系统拥有足够的对换区空间。可以全部从对换区调入所需页面，以提高调页速度。为此，在进程运行前，需将与该进程有关的文件从文件区复制到对换区。

2）系统缺少足够的对换区空间。凡不会被修改的文件都直接从文件区调入；而当换出这些页面时，由于它们未被修改而不必再将它们换出。但对于那些可能被修改的部分，在将它们换出时须调到对换区，以后需要时再从对换区调入（因为读的速度比写的速度快）。

3）UNIX 方式。与进程有关的文件都放在文件区，因此未运行过的页面都应从文件区调入。曾经运行过但又被换出的页面，由于放在对换区，因此下次调入时应从对换区调入。进程请求的共享页面若被其他进程调入内存，则无须再从对换区调入。

分页存储管理的特点

- 分页存储管理的特点：
  - 非连续的存储分配，提高了存储空间的利用率
  - 实现虚拟存储技术
  - 页表占用额外的存储开销
  - 请求分页存在抖动现象，降低CPU的利用率
  - 分页破坏了程序的完整性

分页存储管理的特点

- 反向页表
  - 目的：减小页表占用的总存储空间
  - 思路：以块号（而非页号）为序建立反向页表。

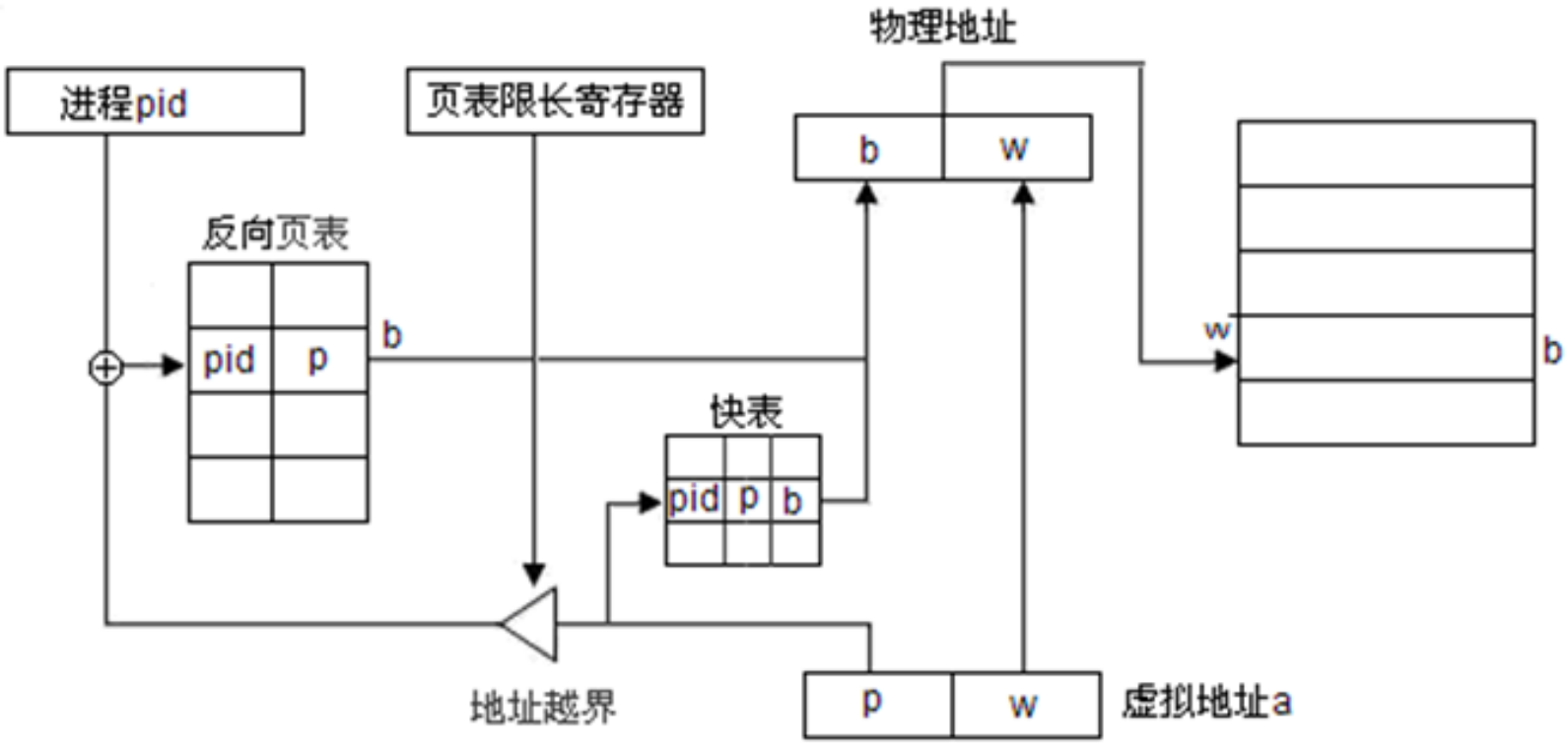
进程	页号	块号	P
A	0	1	1
	1	0	1
	2		0
B	0	2	1
	1		0



块号	页号
0	A1
1	A0
2	B0

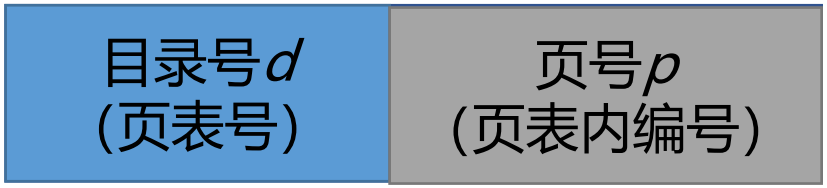
分页存储管理的特点

- 反向页表
  - 重定位



分页存储管理的特点

- 多级页表
  - 目的：减小页表占用的物理存储空间
  - 思路：对一个完整的页表划分为若干个小页表，实现对页表本身的虚拟存储。



高$n-k-t$位高$n-k$位　低t位

每个记录2^t个页！！！

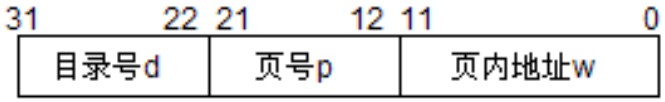


图5-35 32位CPU的虚拟地址结构

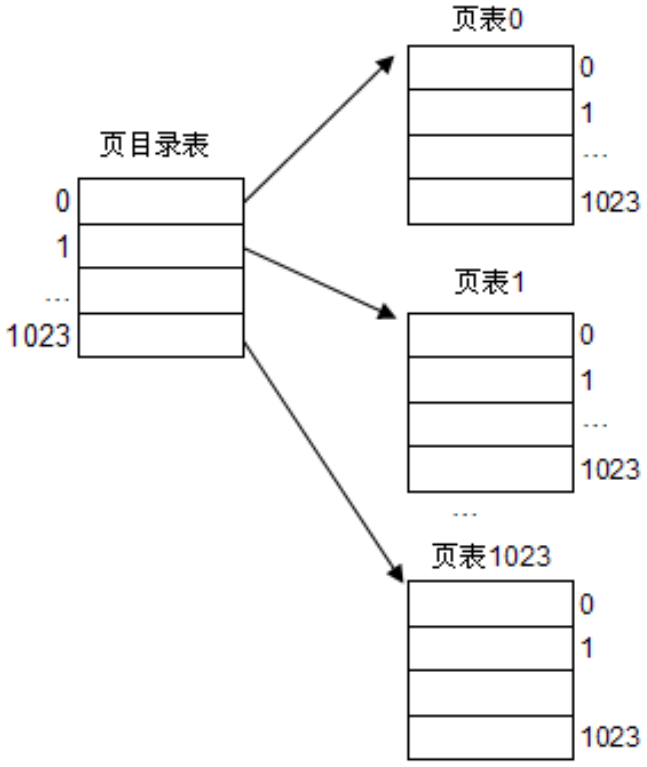
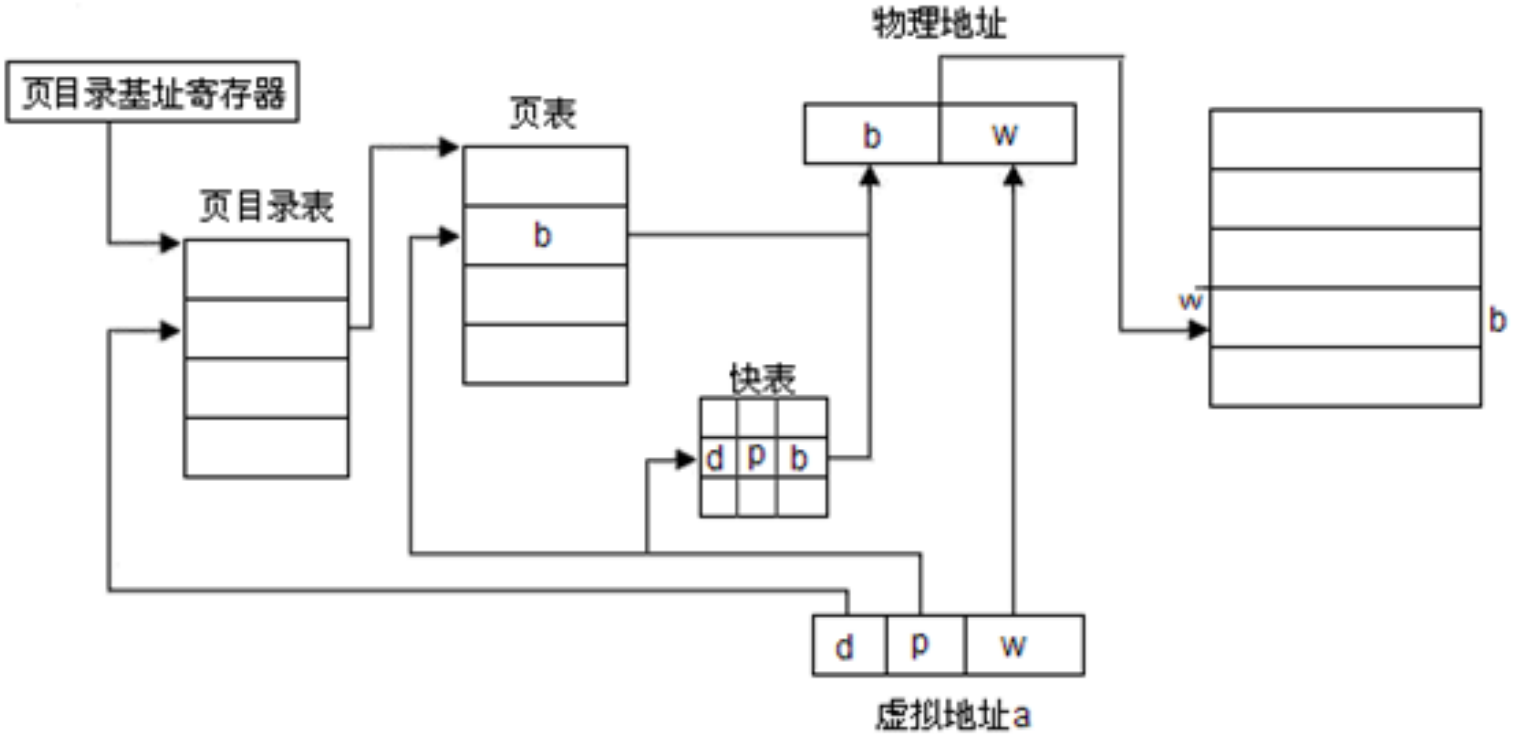


图5-36 一个进程的页目录表及其页表的关系

分页存储管理的特点

- 多级页表
  - 重定位



分页存储管理的特点

- 页长度与存储开销的关系
  - 页数/页表开销/缺页中断次数
  - 每次缺页中断IO操作读写信息量及时间/每个进程的平均内碎片
- 假设系统中页长度为p，系统所需处理的进程大小为s，页表中每个表项的长度为e，那么，
  - 每个进程需要占用的页表空间约为es/p
  - 每个进程产生约$p/2$大小的碎片
  - 于是，每个进程的额外存储开销：

$$f(p) = es/p + p/2$$

  - 当 $p = \sqrt{2es}$ 时，f取最小值。

分页-作业1

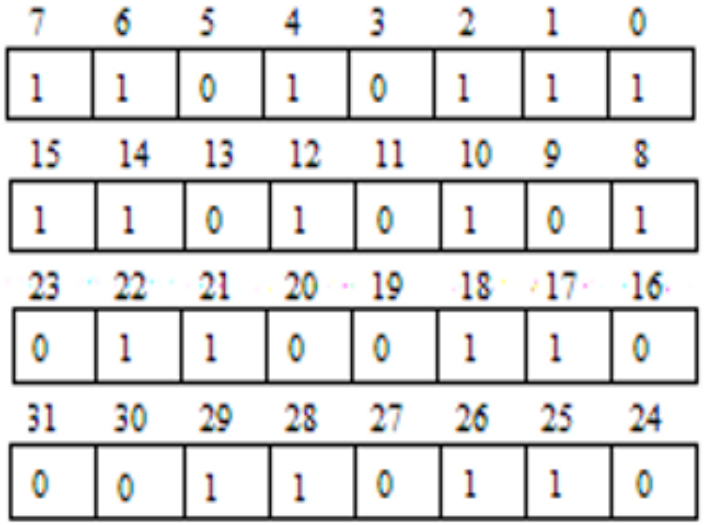
2．静态分页存储管理，假定虚拟寄存器的位数为 16，其中低 9 位用于表示页内地址，进程 A 的页表如下，请给出虚拟地址 2000 的重定位过程。

页号	块号
0	1
1	3
2	129
3	325

进程A页表

分页-作业2

2．在某静态页式存储管理中，已知内存共的 32 块，块长度为 2K，当前位示图如下图所示，进程 P 的虚拟地址空间大小为 18168。



问：

(1)进程 P 共有几页？

(2)根据所给的位示图，给出进程 P 的页表。

(3)给定进程 P 的虚拟地址： 9800(十进制)和 0x68BF(十六进制)，根据(2)的页表，分别计算对应的物理地址。

分页-作业3

3. 某系统采用请求分页内存管理方式。一个进程运行中依次访问页号 0、3、2、3、1、2、3、4、7、6、4、3、5、6。假定系统分配 3 个内存块给该进程，实行局部页面调度，则

（1）给出最久未使用（LRU）算法下的缺页次数和依次淘汰的页号，并写出计算 过程。

（2）给出二次机会算法下的缺页次数和依次淘汰的页号，并写出计算过程。

分页-作业4

3. 某系统采用请求分页的内存管理方式，块大小为 2K，某时刻内存位示图如下

F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	1	1	0	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1

现有一进程开始运行，其逻辑空间大小为 10K。回答下列问题：

（1）此刻内存有多少空闲块？该进程共多少页？

（2）系统采用局部页面替换策略，二次机会替换算法。若该进程依次访问页面 0、 3、1、2、1、3、4、1、0、3，则共发生几次缺页中断？依次淘汰的页面是哪些？

（3）在（2）的基础上，现访问该进程的逻辑位置 0x98C，则重定位到的物理地 址是什么？（假设位示图所表示的用户空间地址起点为 0，且当存在多个空闲块 时，内存分配按空闲块块号从小到大依次分配）

计算机操作系统

第5章 存储器管理(3)

分段存储管理

- 将进程按其内部结构划分为若干段，将每段看作一个独立的逻辑空间，段的内部独立编排地址（段内地址）。
- 使用二维逻辑地址，即“段号:段内地址”的形式。

段0 (DS)	0~1 2~8	10 Hello.	
段1 (CS)	0 1~4 5~8	???? ???? ????	(将[DS:1]压栈;) (cout << [DS:0~DS:1];) (cout << [DS:2~DS:8];)
段2 (SS)	0~…	堆栈	

分段存储管理

- 将进程分段，并按段编址，便于实现：
  - 进程的段内动态扩展（打破整道程序虚拟地址的连续性）
  - 模块共享/动态链接（保留程序结构信息的完整性）
  - 存储保护（区分数据段和代码段）

  - 内存动态分区（类似于可变分区存储管理）
  - 非连续存储分配（各段可装入到不连续的物理内存区域中）
  - 虚拟存储技术（按段调入调出）

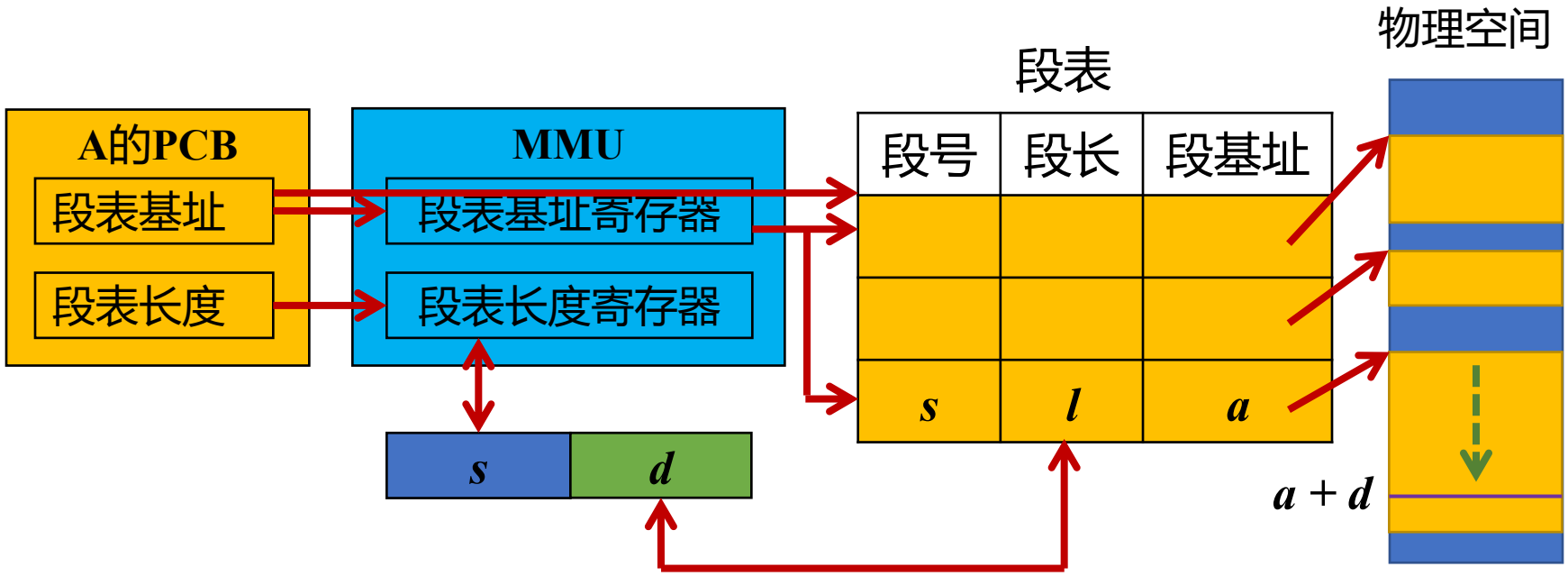
分段存储管理的实现关键—数据结构/分配回收

- 1.内存分配采用类似可变分区存储管理的方式来记录内存空闲区，可采用可变分区的数据结构：空闲区链表
- 2.为记录进程中各段的信息，必须维护数据结构——段表
  - 段表的结构：段号、段长度、所在内存分区起始地址（段基址）、外存地址、存取控制信息、中断位P、访问位A和修改位M等。
  - 每一个进程都对应一个段表，通常，段表是存储在内存的系统区中，作为内核的关键数据结构之一。
- 分配回收：类似于可变分区存储管理

分段存储管理的实现关键—重定位及存储保护

❖ 分段存储管理中的重定位：

进程的段表始址和段表长度被放入PCB中。



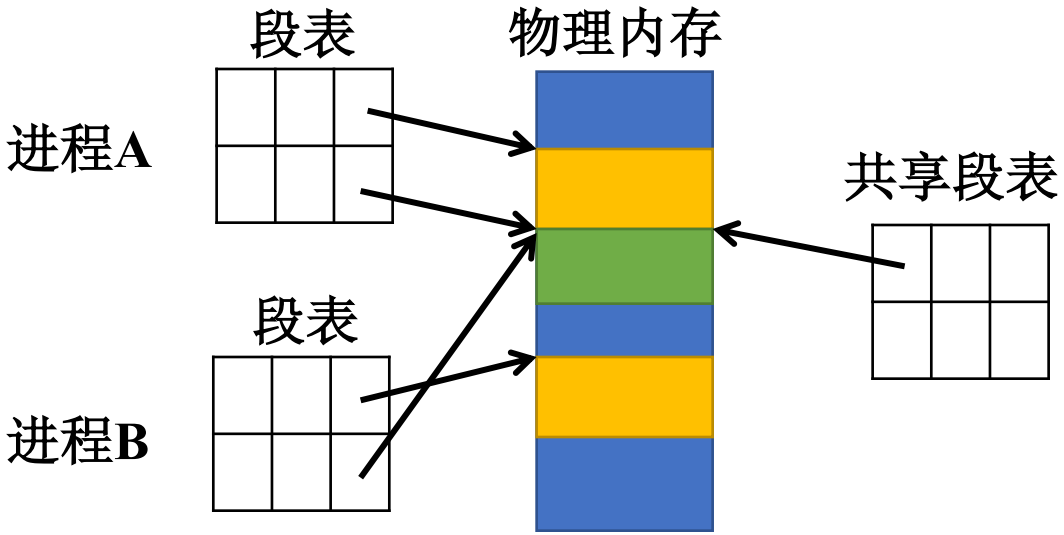
若重定位时所需要的段不在内存中（P = 0），MMU将发出缺段中断。

分段存储管理的实现关键—重定位及存储保护

- 分段存储管理中的重定位：
  - 进程创建时，建立段表，记录段表基址、段表长度。
  - 进程变为运行状态时，将段表基址和段表长度写入MMU的寄存器中。
  - 进行内存访问时， MMU检查段号是否越界（超出段表长度）
  - MMU按照段号查找段表，并检查段内地址是否越界（超出段长）
  - MMU根据段基址与段内地址确定物理地址
  - 若重定位时所需要的段不在内存中（P = 0），MMU将发出缺段中断。

分段存储管理的实现关键—段的共享

- 分段存储管理可实现段（模块）的共享：
  - 某些段可以提供给多个进程共享使用，称作“共享段”，系统维护一个共享段表，来记录这些段。
  - 当某个进程需要使用共享段时，只需将此段的信息列入自身的段表中即可。
  - 共享段可以是数据段也可以是代码段。若共享代码段，一般为只读的纯代码，称作可重入代码。



内存的逻辑扩展

- 几种对内存进行逻辑扩展的方法比较：

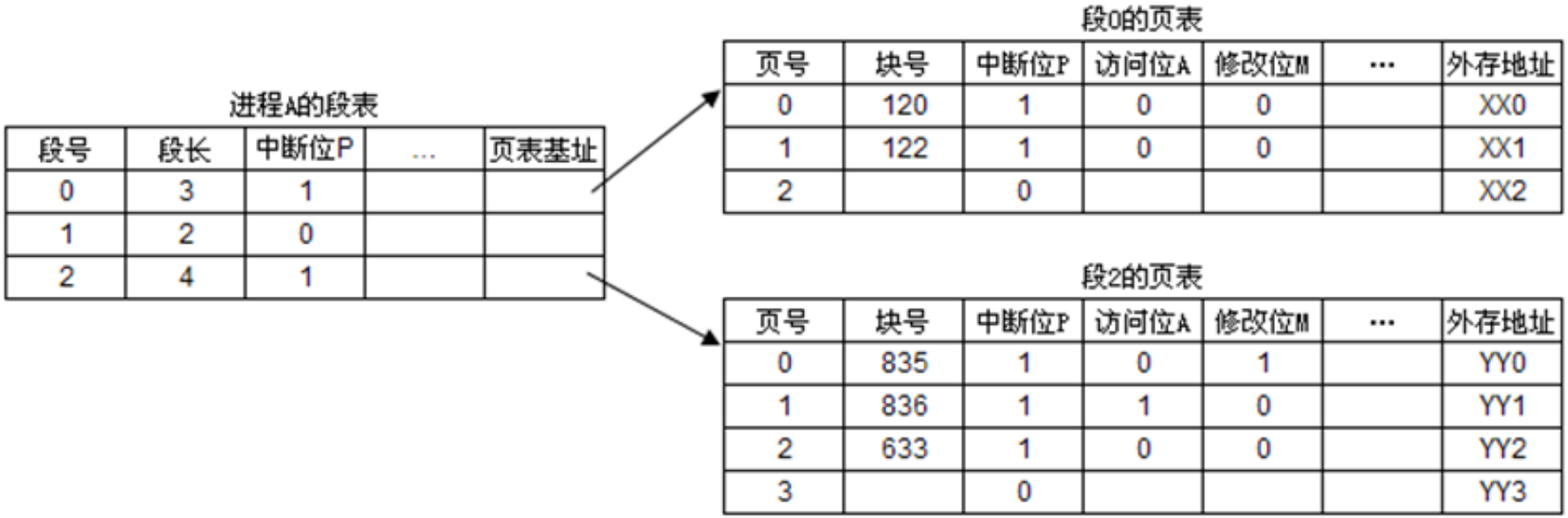
	对换技术	覆盖技术	动态分页	分段管理(非分页)
控制者	操作系统	用户程序		
编程透明度	透明	不透明		
调度粒度	整个进程	子程序		
单个程序空间	仍然受限	可能突破限制		
并发执行程序数	不受限制	仍然受限		

段页式存储管理

- 为了发挥分页管理的优势（最灵活的内存扩展）和分段管理的优势（逻辑空间扩展、模块共享），将两者结合，提出段页式存储管理。
  - 内存分块
  - 进程（程序）分段
  - 段分页
  - 页装入块中（可不连续）
  - 按段或页调入/调出，实现虚拟存储

段页式存储管理的实现关键—数据结构

- 段页式存储管理中，对于一个进程，需要维护一个段表和多个页表。段表中的每项记录一个段的信息，以及这一段的页表位置；页表中的每项记录一个页的信息。



段页式存储管理的实现关键—重定位

- 段页式存储管理的重定位过程：

